



UNIVERSITY OF PADOVA

DEPARTMENT OF DEPARTMENT OF INFORMATION ENGINEERING

MASTER THESIS IN BIG DATA MANAGEMENT AND ANALYTICS

DATA-AWARE PROCESS SIMULATION UNDER PHYSICAL CONSTRAINTS: A CONTAINER TERMINAL TRUCK OPERATIONS CASE STUDY

SUPERVISOR

PROF. MASSIMILIANO DI LEONI
UNIVERSITY OF PADOVA

CO-SUPERVISORS

MARCEL PETERSEN
HHLA - PORT OF HAMBURG
PROF. ESTEBAN ZIMANYI
UNIVERSITÉ LIBRE DE BRUXELLES

MASTER CANDIDATE

JULE GRIGAT

STUDENT ID

2185651

ACADEMIC YEAR

2025-2026

TXAPELA BURUAN 'TA IBILI MUNDUAN: *RUE DE L'ÉTÉ, CARRER DE VIOLANT D'HONGRIA REINA*
D'ARAGÓ, VIA LUIGI EINAUDI AND VIA FRANCESCO BELTRAME.

Abstract

Container terminals use simulation to test operational changes without disrupting live operations, but these models are usually built and calibrated by hand. Data-aware process simulation aims to learn executable models from event logs. Evidence from physical logistics systems is still limited.

This thesis evaluates the approach for truck operations at HHLA Container Terminal Burchardkai. Two months of data were transformed into an event log with 89,460 truck visits and split temporally into training and test periods. The white-box ProSiT framework was used to compare a distribution-only model with a model using decision rules and workload features. The first Inductive-Miner model allowed impossible parallel behaviour within one truck visit. The final pipeline instead combines a manually implemented prefix-trie contract with automatically discovered timing, arrival, routing, and resource parameters.

The prefix trie covered 99.944 % of the 17,892 held-out visits. This shows a highly repetitive activity language, not structural generalisation. The final model reproduced arrival spacing closely and parts of yard service times, but mean truck turnaround was 9.31 minutes too short and the 90th percentile was 19.20 minutes too short. Before calibration, the model with rules and workload features produced a 56 % higher turnaround-distance error than the distribution-only model. Both control flow and RMG capacity required domain corrections. Two what-if experiments were transparent and repeatable, but their mean-turnaround confidence intervals included zero. The model also lacked the spatial state needed for physical counterfactual claims.

Data-aware process simulation can provide an auditable historical model and a useful diagnostic tool for container-terminal operations. Direct operational decision support requires domain constraints and richer models of location, resources, and queueing.

Contents

Listing of figures

Listing of tables

Listing of acronyms

BPMN	Business Process Model and Notation
CHE	Container Handling Equipment
CRM	Customer Relationship Management
CTB	HHLA Container Terminal Burchardkai
DES	Discrete Event Simulation
DFG	Directly-Follows Graph
ERP	Enterprise Resource Planning
GOS	Gate Operating System
HHLA	Hamburger Hafen und Logistik AG
HO ₂	Manual yard handling area
KPI	Key Performance Indicator
LL	Leerlager (empty-container yard)
MES	Manufacturing Execution System
OCR	Optical Character Recognition
PM	Process Mining
RMG	Rail-Mounted Gantry Crane
TAS	Truck Appointment System
TCS	Terminal Control System
TEU	Twenty-foot Equivalent Unit
TOS	Terminal Operating System
TTT	Truck Turnaround Time

VC Van Carrier
WfMS Workflow Management System
XES eXtensible Event Stream
YMS Yard Management System

1

Introduction

Container terminals connect maritime and hinterland transport networks. Higher container volumes and larger vessels require terminal operators to make many tactical and operational decisions under uncertainty. These decisions concern truck dispatching, yard-block allocation, resource use, congestion, equipment failures, and temporary infrastructure outages. Experiments in an operating terminal are often costly, disruptive, or not feasible. Simulation offers a way to assess decisions without interrupting terminal operations.

1.1 MOTIVATION

Simulation is used for decision support in container terminal planning and operations research. It can assess infrastructure investments, operational policies, scheduling approaches, truck appointment systems, and terminal redesigns before implementation. Traditional approaches mostly use manually developed Discrete Event Simulation (DES) models, agent-based models, or digital twins. Building and maintaining these models requires domain knowledge, manual work, and calibration [? ? ? ?].

Process Mining research has studied how simulation models can be built from event logs. Earlier work showed that control-flow behaviour, activity durations, arrival patterns, resource al-

locations, and routing probabilities can be discovered from process data and used in executable simulation models [?]. *Data-aware process simulation* extends this idea. It uses context attributes, case information, process states, resource characteristics, and decision logic [? ? ? ?].

Machine-learning and deep-learning methods can learn complicated behavior, but their results are often difficult to inspect, modify, and validate. Hybrid and white-box approaches combine interpretable models with data-driven behaviour discovery [? ?]. ProSiT uses probabilistic white-box predictors to build simulation models that can be inspected and configured for what-if analysis [?].

Evidence on automatically discovered simulation models in logistics systems is still limited. Existing evaluations mainly concern business processes in healthcare, financial services, and administration [? ?]. Container terminal truck operations involve physical resource constraints, congestion, infrastructure dependencies, spatial interactions, and changing process contexts. It is not yet clear how well a discovered data-aware simulation model can reproduce these conditions and support decisions. This thesis studies its fidelity, structural applicability, and limitations using event logs from real container terminal truck operations.

1.2 PROBLEM STATEMENT

Container terminal operators make decisions about capacity, truck handling, yard use, policies, and infrastructure changes. Experiments in a real terminal are often not feasible or involve operational risk, so these decisions are often studied with simulation. Most container-terminal simulation models are built manually. Their construction and maintenance require domain knowledge and modelling work. They may include subjective assumptions about process behaviour, resource interactions, and routing logic [? ?].

Data-aware process simulation discovers simulation models from historical event logs. This may reduce modelling work and keep the model closer to the recorded process [? ? ?]. Before such a model can support decisions, it must reproduce observed operations, remain configurable, and produce stable results. Earlier studies have shown that data-aware simulation can be applied to business processes [? ?]. Its use in logistics systems with congestion, competing resources, and context-dependent behaviour is less clear. This thesis evaluates the model's fidelity, use, and limitations for container-terminal truck operations.

1.3 RESEARCH OBJECTIVE AND CONTRIBUTIONS

The objective of this thesis is to evaluate the fidelity, structural applicability, and limitations of data-aware process simulation for container terminal truck operations. The study investigates how far a simulation model derived from historical event data using ProSiT can reproduce unseen truck-handling behaviour at the Container Terminal Burchardkai (CTB), and where the transfer of process-simulation discovery to a physical logistics environment reaches its limits. Trustworthiness motivates this evaluation, but it is not treated as a property that can be established from a single historical case study.

More specifically, the thesis evaluates whether the control-flow representation derived from the training data remains applicable to a later held-out period while respecting the physical sequence of a truck visit. It then assesses the historical fidelity of the resulting simulator, examines whether increasing the contextual expressiveness of the ProSiT configuration improves this fidelity, and investigates how the data-aware model responds to transparent operational interventions. Together, these analyses are intended to distinguish between reproducing historical behaviour, constructing an executable simulation model, and supporting credible what-if reasoning.

The thesis contributes an empirical evaluation of data-aware process simulation in container terminal truck operations. First, it examines the transfer of a recent process-simulation discovery approach to a logistics setting characterised by physical sequencing, shared resources, temporal variation, and congestion. Second, it identifies methodological challenges that arise when automatically inferred process and resource abstractions do not fully correspond to known physical constraints, requiring explicit domain-constrained adaptations. Third, it compares different levels of contextual model expressiveness on unseen temporal data rather than assuming that additional data awareness necessarily improves simulation quality. Finally, two operational interventions are used to investigate both the capabilities and the boundaries of the resulting white-box model for what-if analysis and decision support.

The empirical study is based on operational CTB truck-event data and combines process analysis, simulation-model generation, temporal hold-out validation, multi-seed simulation, and controlled scenario experiments. The corresponding research questions specify how these objectives are evaluated in the remainder of the thesis.

1.4 RESEARCH QUESTIONS

To achieve the research objective, the thesis addresses the following research questions:

1. RQ1: To what extent can a ProSiT simulation model discovered from historical CTB truck-operation data reproduce the observed behaviour of an unseen temporal hold-out?
2. RQ2: To what extent can the discovered process structure represent held-out CTB truck-handling behaviour while respecting the physical sequential constraints of a truck visit?
3. RQ3: How does increasing the contextual expressiveness of the simulation model affect held-out simulation fidelity in the CTB truck process?
4. RQ4: How does the discovered simulation model respond to transparent operational interventions, and what do these responses reveal about its suitability and limitations for what-if decision support?

1.5 THESIS OUTLINE

Chapter 2 establishes the theoretical and methodological background of the thesis. It first introduces container-terminal operations and truck handling as the application domain before explaining how operational information systems can be transformed into event logs for Process Mining. The chapter then reviews Process Mining, Business Process Simulation, simulation discovery, and the development towards data-aware and white-box simulation approaches. Particular attention is given to ProSiT and to the validation of automatically derived simulation models. The chapter concludes by defining the research gap concerning the transfer of these approaches to physical logistics systems with spatial and resource constraints.

Chapter 3 introduces the HHLA Container Terminal Burchardkai as the empirical case study and describes the operational data used in the analysis. It explains how records from several terminal systems are cleaned, matched, and transformed into a process-oriented truck event log. The chapter defines the case and activity structure, the lifecycle timestamps used for service-time modelling, and the contextual attributes required for data-aware simulation, including

demand, utilisation, container, and target-area information. It concludes with the structure and descriptive characteristics of the final simulation-ready event log.

Chapter 4 defines the experimental and evaluation framework used to answer the research questions. It introduces the temporal train/test split and describes the control-flow discovery procedure, including the initial Inductive-Miner model, the subsequent domain check, and the domain-constrained prefix-trie representation used by the final simulator. The chapter then defines the ProSiT parameter-discovery configurations with different levels of contextual expressiveness and documents the executable parameter bundles used in the experiments. In addition, it specifies the Monte-Carlo replication protocol, the distributional fidelity measures, the two what-if interventions, and the transition-duration sensitivity analysis. The final section documents the technical adaptations required to apply ProSiT to the CTB event log reproducibly.

Chapter 5 presents the empirical findings following the logic of the four research questions. It first examines the temporal train/test split and the stability of the observed control-flow language in the held-out period. It then evaluates the historical fidelity of the final simulation baseline and compares the retained ProSiT configurations to assess the effect of additional contextual expressiveness. Further diagnostic analyses examine the predictive signal contained in the available context attributes, the strong temporal structure of truck arrivals, the discovered resource capacities, and the interpretation of model-derived pre-service delays. The chapter finally reports the two what-if experiments and discusses the methodological limitations that constrain their interpretation.

Chapter 6 brings together the empirical findings and answers the four research questions. It evaluates what the case study demonstrates about the transfer of data-aware process simulation to container-terminal truck operations and distinguishes historical fidelity and model executability from the stronger requirements of physical what-if decision support. The chapter concludes by outlining directions for future work, particularly concerning richer spatial and resource-state representations, alternative event-log designs, and further validation in other operational settings.

2

Background and Related Work

2.1 CONTAINER TERMINAL OPERATIONS

Container terminals connecting maritime and hinterland transport networks are also called seaport Terminals. They link deep-sea vessels with road, rail, and inland waterway transport. Large container vessels can carry more than 20,000 containers, so terminals must handle large cargo volumes while keeping turnaround times short [?]. Since the 1960s, standardised containers have become the main unit of global freight transport. Container throughput is measured in Twenty-foot Equivalent Units (TEU): a 20-foot container equals one TEU and a 40-foot container equals two TEU [?]. Larger vessels and higher trade volumes place terminals under pressure because many processes and resources must be coordinated at the same time. Operational decisions affect vessel berth times, truck turnaround times, terminal throughput, and service quality [? ?]. This thesis studies the HHLA Container Terminal Burchardkai (CTB) in Hamburg, which has an annual throughput of averagely 3 mio TEU, with all HHLA Container Terminals in Hamburg having a total throughput of averagely 6.3 mio TEU annually.

2.1.1 CONTAINER TERMINALS AS COMPLEX LOGISTICS SYSTEMS

Container terminals are complex logistics systems because many resources, transport modes, and operational processes interact during daily operations [? ?].

CONTAINER FLOWS

Within a seaport terminal, containers move between transport modes and storage locations. A container may arrive by vessel, truck, or rail and later leave by one of these modes. The terminal coordinates flows between the seaside and landside [?]. Hartmann distinguishes deep-sea vessels, feeder vessels, trucks, and rail as the four main transport modes. Their combinations create many possible paths through the terminal [?]. Most containers are stored in the yard before they leave the terminal. Terminals must manage both transport processes and storage allocation [?]. Figure 2.1 shows the basic connection between the waterside, the yard, and hinterland transport.

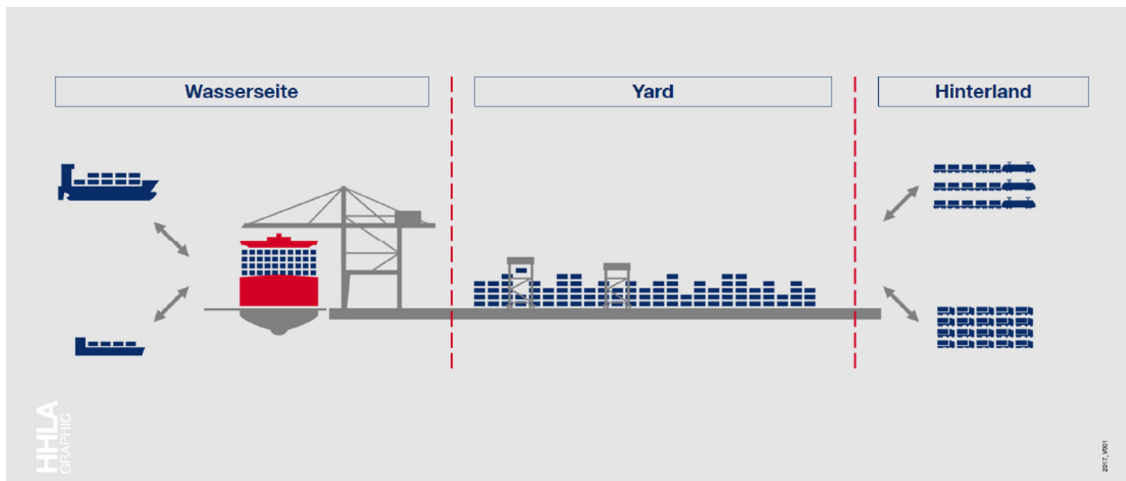


Figure 2.1: Container flows between the waterside, yard, and hinterland. Source: Hamburger Hafen und Logistik AG [?].

And Explanation, that Hamburg is a convenient hub for the European Hinterland traffic, as its railway route to Eastern Europe is shorter than from other Western European harbours like Rotterdam.

YARD COMPONENTS AND RESOURCES

Container terminals consist of several connected operational areas. Bett et al. distinguish the quayside, yard, and landside. The quayside includes berths and quay cranes for loading and

unloading vessels. The yard stores containers between transport steps, while the landside connects the terminal to trucks and rail operations [?]. This thesis focuses on the CTB yard, which has three areas:

1. Automatic storage Blocks: Most containers are stored in automatic blocks operated by Rail Mounted Gantry Cranes (RMGs). The CTB operates 22 RMG blocks with 2,231 operational TEU slots and three RMGs per block between the waterside and landside. [total of 40.000 operative TEU in Block] up to 5 high due to operational moves.
2. Manual Yard: Containers can be stored in a manually operated yard. Its central location allows storage decisions to depend on the assigned vessel berth, yard workload, and equipment availability. Manual handling equipment moves the containers.
3. Empty Container Yard: Empty containers are stored in a separate yard. They are often managed as interchangeable inventory within a shipping line's container fleet instead of being assigned to one order. Empty containers without a fixed delivery order can be stored densely. Keeping them separate prevents them from blocking full containers that are linked to transport orders. Reach stackers operate the empty-container yard and can stack containers up to six high.

Trucks can pick up or deliver containers in all three yard areas. At each area, they wait at a transfer point for the assigned handling resource. A resource shortage in one area can delay other terminal operations [?].

INTERDEPENDENCIES BETWEEN TERMINAL PROCESSES

Terminal processes are connected, so a local decision or disruption can affect other parts of the terminal. [For example, delayed vessel arrivals may postpone unloading and change yard occupancy. Yard congestion can reduce crane productivity and increase truck waiting times.] - Mit Fokus, was bedeutet das eigentlich tatsächlich für ein terminal? [? ?]. These interactions are difficult to evaluate with analytical optimisation alone.

2.1.2 TRUCK HANDLING PROCESSES

IMPORTANCE OF TRUCK OPERATIONS

Road transport is part of a container terminal's hinterland connection. Terminals must coordinate many truck arrivals while limiting queues inside the terminal and on public roads in the port area [? ?]. Changing arrival volumes can exceed the capacity of gates, yard equipment, and storage areas. This increases waiting times and reduces terminal performance [? ?].

A TRUCK VISIT PROCESS

Many container terminals use a Truck Appointment System (TAS) to regulate arrivals and reduce congestion. A trucking company reserves a time slot before the visit and states which container transactions it will perform [? ?]. At the automated pre-gate, Optical Character Recognition (OCR) systems identify the container and vehicle and match this information with the booking. [Despite the hourly booking slots, the Depending on the current workload and gate queue lengths, the truck may have Referr to Poisson later in the arrival distribution section] The truck may then wait before the physical gate admits it to the terminal [?]. It drives to the assigned yard location and performs one or more transactions:

1. delivering an export container,
2. receiving an import container,
3. delivering or receiving multiple containers while at the same yard location,

An efficient pattern is the *multi move*. A truck delivers one or more export containers and collects one or more import containers during the same visit. This reduces empty trips, emissions, and congestion, but requires good terminal routing [?]. After all transactions, the truck passes the exit gate and leaves the terminal. The time between entry and exit is the *Truck Turnaround Time (TTT)*, a performance indicator for truck operations [?].

TERMINAL OPERATING SYSTEMS

Container terminals use several information systems to coordinate operations. The Order Management and Terminal Control System (TCS) manages container movements, storage locations, and operational orders. It is connected to other systems, including a Gate Operating System (GOS), yard management systems, equipment control systems, and appointment platforms.

Each system records a different part of the operation. Gate systems record arrivals and departures, while yard systems record container movements and storage assignments. Equipment control systems record the execution of handling orders. A complete truck visit must be reconstructed from several data sources. This thesis mainly uses the Order and Terminal Control System and adds data from other systems to build and check the event log. Chapter 4 describes this data integration.

SOURCES OF OPERATIONAL DELAYS

Truck turnaround time depends on several factors. Common reasons for delay include:

- congestion at entry and exit gates, including effects from road traffic outside the terminal;
- pick-up containers that are blocked by other containers after a long dwell time;
- truck traffic congestion within container yard sections and main routes,
- limited availability of container handling equipment (CHE),
- high occupancy levels within storage blocks,
- resource and priority conflicts between truck and vessel operations,
- unexpected equipment failures or maintenance activities

These factors are connected, so a local disruption can affect other terminal operations. For example, low yard-crane availability or vessel priority can increase truck service and waiting times. Longer queues can then cause yard traffic, reduce gate throughput, and slow hinterland operations [? ?].

OPERATIONAL PERFORMANCE INDICATORS

Truck operations are evaluated with operational key performance indicators (KPIs).

- Truck Turnaround Time (TTT): Total time a truck spends inside the terminal between entry and exit.
- Waiting Time: Time spent waiting for gate processing, yard equipment, or storage area access.

- Yard Utilization: Degree to which storage areas are occupied and operational resources are utilized.

Among these indicators, Truck Turnaround Time is often the main measure of truck service quality because it reflects the time experienced by trucking companies and cargo owners. It is one of four main KPIs used by the HHLA container terminals [? ?].

2.1.3 SIMULATION-BASED DECISION SUPPORT IN CONTAINER TERMINALS

This is now trying to explain Why is there a need for better decision support in container terminals in general, why is simulation a common approach for that purpose, and why are traditional simulation approaches insufficient?

Terminal managers make decisions about truck appointments, resources, equipment schedules, maintenance, and yard operations while the terminal remains active. Because terminal processes are connected, local decisions can change waiting times, resource use, throughput, and service quality. Even experienced terminal managers face uncertainty. Changes to procedures or capacity can create congestion, increase handling times, or reduce service quality. Simulation allows managers and researchers to evaluate such changes without affecting live operations. Common applications include Truck Appointment Systems (TAS), equipment allocation, resource planning, yard management, and storage strategies. Simulation is useful for congestion, bottlenecks, and trade-offs that are difficult to analyse directly. Studies have shown its value for truck turnaround time, resource use, and terminal performance [? ? ?]. Practical use is still limited by model complexity, costs, and missing technical expertise in terminal teams.

Different simulation approaches are used in terminal research. Discrete Event Simulation (DES) is the most common because terminal operations can be modelled as events such as truck arrivals, gate processing, container handling, and equipment use. System Dynamics, Agent-Based Modelling, and traffic simulation are used for other operational and strategic questions [? ? ?]. Traditional simulation projects have limits. Building a model requires domain knowledge, process understanding, and much manual work. Data collection, implementation, calibration, and validation take time. Manual models depend on expert assumptions, which may describe the intended process rather than its actual execution [?].

These differences can reduce the quality of simulation results. Manual models must be maintained and calibrated when processes, workloads, and operating rules change. This work is

costly and requires technical support. At the same time, terminal information systems record large amounts of operational data. Process Mining can use these data to discover, analyse, and monitor processes from their recorded execution. This may reduce manual modelling work and make simulation models closer to the observed process.

2.2 OPERATIONAL DATA AND EVENT LOGS

2.2.1 TRANSACTIONAL INFORMATION SYSTEMS

Woraus und warum entstehen überall überhaupt Eventlogs? Most business processes are supported by information systems. These systems document operational activities, but they are not always designed to analyse a complete process. Examples are Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), Manufacturing Execution Systems (MES), and the Terminal Operating Systems (TOS) used in container terminals. User actions and object changes create records in the underlying databases. Creating, updating, processing, or completing an order leaves information about parts of the process execution. In container terminals, the TOS, Gate Operating System (GOS), and Yard Management System (YMS) record operational transactions. Examples include a truck arrival, a storage assignment, and a container-handling operation. Each record provides information about what happened, when it happened, and which object was involved. Because business processes are executed step by step, recorded transactions form event sequences. These data are the basis for Process Mining, which reconstructs process behaviour from information systems. At HHLA CTB, the systems have grown over time. They do not all record the same level of detail and sometimes use different names for the same facility. The event-log engineering section discusses this issue.

2.2.2 EVENTS, CASES AND TRACES

Process Mining describes a process as a sequence of recorded events. The following terms define this event-log structure.

EVENT An event records an activity at a point in time. Examples from a container terminal are shown in Table 2.1.

Event	Description
OCR gate entry	The truck passes the automatic identification system at the terminal entrance.
Gate check completed	The administrative and security checks at the entry gate are completed.
Yard position reached	The truck arrives at its assigned handling position in the yard.
Container operation completed	The loading or unloading operation at the yard position is completed.
Terminal exit	The truck leaves the terminal after its operations are completed.

Table 2.1: Examples of events in a container terminal truck process.

ACTIVITY An activity describes the type of action that is executed. Multiple events can belong to the same activity while occurring at different times or for different process instances. Table 2.2 gives examples.

Activity	Description
Gate In	Entry checks and authorization of the truck visit.
Container Delivery	Transfer of a container from the truck to the terminal yard.
Container Receive	Transfer of a container from the terminal yard to the truck.
Gate Out	Completion of the exit procedure before the truck leaves the terminal.

Table 2.2: Examples of activities in a container terminal truck process.

CASE A case represents a single process instance. It is defined by the object whose process execution is analyzed. Possible case objects are shown in Table 2.3.

In this thesis, one truck visit is one case.

In traditional event logs, events are often represented by a single timestamp, typically indicating the completion of an activity. Performance-oriented Process Mining and simulation require more temporal information. Table 2.4 shows the two directly observable lifecycle timestamps.

The duration between these two timestamps is the *service time* (or execution time) of the activity. A third temporal concept, *enablement*, describes the moment at which an activity becomes executable according to the process-model state. In process-mining-based simulation

Case object	Description
Customer order	One order is followed from its creation to its completion.
Container	The recorded handling process of one container forms the case.
Truck visit	The recorded process from a truck's arrival to its departure forms the case.

Table 2.3: Examples of objects that may define a process case.

Attribute	Description
Start timestamp	Time at which the execution of the activity begins.
Complete timestamp	Time at which the execution of the activity ends.

Table 2.4: Observable timestamp attributes of an event.

approaches, enablement is typically not an observed input-log attribute but is derived by replaying the event log against the discovered process model [? ? ?]. The interval between model-derived enablement and the observed start timestamp is referred to as *waiting time* in the ProSiT framework [?] and corresponds to a pre-service delay that may include resource contention, queueing effects, and — in domains with spatial separation between activities — physical movement time.

ATTRIBUTES Events may contain attributes that describe the process context. Examples from container terminal operations are shown in Table 2.5.

Attribute	Description
Truck ID	Identifier of the truck involved in the recorded operation.
Container number	Identifier of the container handled during the visit.
Shipping line	Shipping company associated with the container.
Yard block	Storage block in which the container operation takes place.
Container type	Operational or physical category of the container.
Equipment identifier	Identifier of the equipment used for the operation.
Trucking company	Company responsible for the truck visit.

Table 2.5: Examples of contextual event attributes in container terminal operations.

Data-aware Process Mining and simulation methods can use these attributes as model inputs.

TRACE A trace is the chronologically ordered sequence of all events belonging to a specific case. For a truck visit, a trace may appear as follows:

OCR Gate Entry → Gate In → Yard Arrival → Container Delivery → Container
Pick-Up → Gate Out

A trace describes the execution path of one process instance.

2.2.3 EVENT LOGS

An event log groups events into cases and orders them by timestamp. Each event requires at least three attributes:

- Case Identifier,
- Activity,
- Timestamp (at least one temporal ordering attribute).

For simulation-oriented process mining, richer event information is required:

- Start Timestamp,
- Complete Timestamp,
- Resource,
- Contextual attributes (e.g. container type, yard utilisation).

The CTB input log used in this thesis provides start and completion timestamps required for execution-time modelling, while activity enablement is reconstructed during process-model alignment by the simulation-discovery framework (Section 4.5).

Other attributes can add operational context. Table 2.6 shows a simplified event log.

Event logs can be stored in standard formats for exchange between Process Mining tools. The eXtensible Event Stream (XES) format stores events, cases, timestamps, and other attributes in a structured and tool-independent form [SOURCE]. By organising database records into cases and traces, an event log turns raw operational data into input for process discovery, conformance checking, and performance analysis [SOURCE].

Case ID	Activity	Timestamp	Yard block
T001	Gate In	09:13:04	–
T001	Yard Operation	09:21:46	Bo6
T001	Gate Out	09:42:18	–
T002	Gate In	09:17:12	–
T002	Yard Operation	09:30:51	Bo8
T002	Gate Out	09:58:33	–

Table 2.6: Simplified event log example.

2.3 PROCESS MINING FOUNDATIONS

2.3.1 DEFINITIONS

Companies record growing amounts of operational data through systems such as Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), and Workflow Management Systems (WfMS) [SOURCE]. Business Intelligence, Data Mining, and Process Mining analyse these data in different ways. Business Intelligence (BI) uses dashboards, reports, and key performance indicators (KPIs) to describe past and current performance [?].

Data Mining identifies patterns, correlations, classifications, and predictive relationships in datasets. Traditional methods often treat observations as independent records and ignore the order of events [?]. Process Mining connects process models with recorded event data. It uses event sequences to discover and monitor actual process behaviour and to identify bottlenecks or deviations [? ?].

PROCESS PERSPECTIVES

Process Mining studies a process from four perspectives [?].

CONTROL-FLOW PERSPECTIVE The control-flow perspective describes the ordering relations between activities and captures how process instances progress through a process. It focuses on questions such as which activities occur, in which order, and which alternative execution paths exist.

TIME PERSPECTIVE The time perspective focuses on temporal process characteristics such as activity durations, waiting times, throughput times, and bottlenecks. The analysis of times-

tamps contained in event logs enables the identification of performance issues and delays.

RESOURCE PERSPECTIVE The resource perspective provides the organizational dimension of a process. It considers which resources execute certain activities, how work is distributed among employees or other resource types, and how organizational structures influence process behavior.

DATA PERSPECTIVE The data perspective analyzes process-related attributes and case characteristics. Depending on the type of case, attributes can include customer types, object categories or risk classes. Such attributes often influence process decisions and execution paths [?].

Together, these perspectives describe the process structure, timing, resources, and data.

PROCESS DISCOVERY

The objective of process discovery is to construct a process model from an event log without a predefined process model [?]. The output should reproduce the observed behaviour. Depending on the discovery algorithm, it can be a Petri net, process tree, BPMN model, or directly-follows graph [?]. Three common algorithms are [SOURCES]:

- Alpha Miner: One of the earliest process discovery algorithms, capable of identifying causal dependencies and parallel behaviour from event logs.
- Heuristic Miner: An extension of Alpha Miner that incorporates frequency information and is more robust towards noise and infrequent behaviour.
- Inductive Miner: Recursively decomposes event logs using sequence, exclusive-choice, parallel, and loop cuts. It produces a sound block-structured model and can filter infrequent behaviour [?].

Process discovery shows how a recorded process was executed, rather than how it was expected to operate [?].

PREFIX TREES AND THE OBSERVED TRACE LANGUAGE. A prefix tree, or *trie*, is a rooted tree representation of a set of sequences [?]. Let an event log L be a multiset of traces over activities $\mathcal{A}$. The trie has one node for every observed prefix

$$\text{Pref}(L) = \{\epsilon\} \cup \{\langle a_1, \dots, a_k \rangle \mid \exists \langle a_1, \dots, a_n \rangle \in L, 1 \leq k \leq n\}, \quad (2.1)$$

where ϵ denotes the empty prefix. An edge labelled a connects prefix p to $p \cdot \langle a \rangle$ whenever that continuation occurs in the log. Traces with the same history share one path up to their first different activity; nodes and edges may be annotated with occurrence frequencies or data distributions. PRETSA uses precisely this property in process mining: each node groups events of the same activity whose traces share the same activity prefix [?].

Without pruning or smoothing, a prefix trie accepts exactly the observed sequence variants. It therefore memorises the recorded trace language rather than discovering a compact or general process model. Fitness and precision on its construction log are expected to be one by design and are not evidence of discovery quality. The trie does not infer unseen combinations, loops, or concurrency. An Inductive-Miner model makes the opposite trade-off: its block-structured cuts abstract from individual variants and can admit behaviour not present verbatim in the log [?].

The useful empirical question for a raw trie is therefore not whether it fits its training log, but whether its memorised language still covers a later period. High held-out coverage means that observed sequence variants are strongly repetitive and stable over time; low coverage means that the strict language is unsuitable outside the construction sample. A trie can be useful as an auditable execution guardrail, but it should not be presented as a general discovery of the underlying process.

Figure 2.2 illustrates the two most common output representations of a modern discovery algorithm on the same event log. The process tree in Figure 2.2a decomposes the process into hierarchically nested operators (*seq* for sequence, *xor* for exclusive choice, *and* for concurrency, *loop* for cyclic behavior). The BPMN model in Figure 2.2b is the industry-standard graphical notation for the same behavior and is derived from the process tree by mapping each operator to a BPMN construct (exclusive gateway, parallel gateway, task). Petri-net representations of the same behavior are structurally equivalent but use a different graphical vocabulary (places and transitions); a concrete Petri-net example on the CTB event log is discussed in Section 5.2.

CONFORMANCE CHECKING

While process discovery focuses on constructing process models, conformance checking focuses on comparing observed process executions with an existing process model [?]. The objective is to identify deviations between the discovered model and the observed reality. Resulting deviations can indicate compliance violations, process inefficiencies, exceptional cases, or outdated process documentation. The quality of discovered process models is commonly evaluated us-

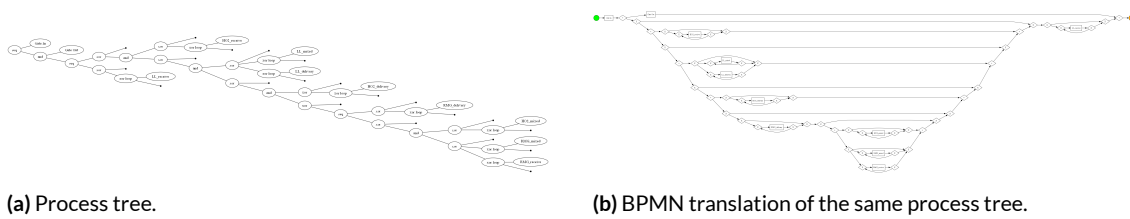


Figure 2.2: Two representations of the same discovered control flow (inductive miner on the CTB training log). The process tree exposes the hierarchical operator structure; the BPMN translation preserves the same behavior using the industry-standard graphical vocabulary. Both models describe an identical set of allowed traces and are used interchangeably in the process mining literature [?].

ing four criterias [? ?]:

- **Fitness:** Measures how well the process model can reproduce the observed behavior in the event log. A model with high fitness allows most observed traces to be replayed without deviations.
- **Precision:** Measures how much additional behavior the model allows beyond the behavior observed in the event log. A precise model avoids allowing many unrealistic or unobserved execution paths.
- **Generalization:** Measures whether the model captures the underlying process behaviour instead of only reproducing the traces in the event log. A model should allow plausible behaviour that was not observed in the available sample.
- **Simplicity:** Measures the complexity of the process model. If several models describe the observed behaviour well enough, the simpler model is usually preferred because it is easier to interpret and analyse.

These criteria provide a balanced framework for evaluating process model quality and form the basis of many process-mining evaluation approaches.

A*-BASED OPTIMAL ALIGNMENT. An alignment relates an observed trace to an executable run of a process model. Formally, it is a sequence of three move types: a *synchronous move* consumes an event and fires a transition with the same visible label; a *log move* consumes an event without a corresponding model transition; and a *model move* fires a model transition without consuming an observed event. A cost function normally assigns zero cost to synchronous moves and

positive, configurable costs to deviations. An optimal alignment λ^* is

$$\lambda^* = \arg \min_{\lambda \in \Lambda(\sigma, N)} c(\lambda), \quad (2.2)$$

where $\Lambda(\sigma, N)$ is the set of legal alignments between trace σ and process model N [? ?].

The qualifier “A*-based” refers to the search procedure, not to a different kind of alignment. A* explores the product state space of the current trace position and the current model state (for a Petri net, its marking). It orders candidates by $f = g + h$, where g is accumulated move cost and h is an admissible lower bound on the remaining cost. With an admissible heuristic, the first complete solution is cost-optimal [?]. The resulting event–transition correspondence supports conformance diagnostics and timestamp replay for performance analysis [?]; it does not turn a model-derived enablement instant into an independently observed timestamp.

PROCESS ENHANCEMENT

Process Mining includes process discovery, conformance checking, and process enhancement [?]. Discovery creates a process model from an event log. Conformance checking compares observed behaviour with a model. Process enhancement adds information from event data to an existing model. Examples include waiting times, activity durations, resource assignments, bottlenecks, and workload. This helps users understand the process and find possible improvements.

2.4 BUSINESS PROCESS SIMULATION

2.4.1 FUNDAMENTALS OF BUSINESS PROCESS SIMULATION

Business Process Simulation (BPS) uses executable models to study process behaviour in a virtual environment. Organisations can run these models under controlled conditions without disturbing live operations [?]. Simulation can estimate throughput times, waiting times, resource use, service levels, and costs before a change is implemented. It is often used for *what-if analysis*. Examples include changing the number of resources, routing rules, workload, or operational policies. The results of different scenarios can then be compared [?]. BPS is used in manufacturing, healthcare, logistics, supply chains, and business-process management. Process Mining allows such models to be discovered and calibrated from recorded process executions

[? ?].

2.4.2 BUSINESS PROCESS SIMULATION MODELS

A simulation model is a simplified representation of a process. Most Business Process Simulation models describe the following parts [? ?].

CONTROL FLOW

Control flow defines which activities exist, their possible order, and the alternative execution paths. It can be represented by Petri nets, BPMN models, process trees, or workflow graphs [? ?].

ARRIVAL PROCESS

The arrival process defines how new cases enter the system. Examples are customer requests, orders, applications, and service tickets. Statistical distributions often model the time between arrivals. Arrival volumes affect waiting times and resource use [? ?].

RESOURCES

Resources execute process activities. They can be employees, machines, vehicles, cranes, or other equipment. Resource models describe availability, capacity, capabilities, schedules, and allocation rules. Limited resources can create competition between cases [? ?].

ACTIVITY DURATIONS

The timing perspective describes processing times, waiting times, and case durations. Probability distributions fitted to historical observations can represent these durations [? ?].

DECISION LOGIC

Business processes contain decision points with alternative paths. A model's decision logic determines which path is selected during simulation. Traditional models use static branching probabilities. Data-aware approaches can instead condition the decision on context, process attributes, or the current operational state [? ?].

Together, control flow, arrivals, resources, timing, and decision logic define the simulation model. Both manual modelling and simulation discovery use these parts.

2.4.3 SIMULATION-BASED DECISION SUPPORT

Container terminal demand changes over time, resource capacity is limited, and operational processes depend on each other. Managers make decisions about equipment, workforce, storage, truck appointments, and capacity while the terminal remains active. Tests in the live terminal can affect service quality and throughput. Simulation allows alternatives to be assessed before they are implemented [? ? ?]. Applications include:

- evaluation of truck appointment policies,
- resource and workforce planning,
- capacity planning and investment decisions,
- yard layout and storage strategies,
- yard crane allocation policies,
- congestion analysis and bottleneck identification,
- equipment scheduling and maintenance planning.

Simulation supports what-if analysis by comparing scenarios under the same conditions. Measures can include truck turnaround time, resource use, throughput, waiting time, and service levels. This is useful in container terminals because a change in gate, yard, or vessel operations can affect other areas [? ? ?]. Truck-operation studies have assessed Truck Appointment Systems (TAS), gate capacity, yard use, and equipment allocation [? ?].

2.4.4 SIMULATION MODEL CALIBRATION

Calibration estimates or adjusts simulation parameters to describe the observed operating conditions. Typical tasks include fitting arrival and duration distributions, estimating resource capacities, and learning routing probabilities or context-dependent decision rules from historical observations [? ?]. In simulation discovery, many of these steps form part of the automated discovery procedure itself. Calibration must still be distinguished from validation: agreement with the data used to fit a model shows in-sample fit, whereas an independent assessment requires data that did not determine those parameters. Section 2.7 develops the corresponding verification and validation concepts.

2.5 SIMULATION DISCOVERY AND DATA-AWARE SIMULATION

The previous sections introduced Business Process Simulation and event logs. Simulation discovery connects both by estimating simulation behaviour from recorded process executions. The estimated parameters are integrated into an executable process model [? ? ?]. Research has moved from basic stochastic parameters towards context-dependent and data-aware behaviour.

2.5.1 FROM SIMULATION DISCOVERY TO DATA-DRIVEN SIMULATION

Rozinat et al. proposed an early approach for deriving simulation models from event logs [?]. It extracts control flow, timing, resource assignments, arrivals, and routing probabilities from historical executions.

Later research extended this idea. Solti et al. [?] study stochastic Petri nets with flexible delay distributions. Camargo et al. [?] proposed a framework that discovers process structure, activity durations, resource pools, calendars, and arrival behaviour from event logs. Simod further automates the discovery and calibration of simulation models [?].

Another research branch studies generative process models. Camargo et al. [?] compare conventional data-driven simulation with deep-learning models. These models can learn dependencies beyond separately fitted distributions, but they are harder to inspect and configure. This issue is discussed in Section 2.6.

Research has moved from extracting standard simulation parameters to learning relations between process state and simulated behaviour [? ? ?].

2.5.2 DISCOVERY OF SIMULATION BEHAVIOR

Simulation discovery still models the parts introduced in Section 2.4.2, but later methods represent them in different ways.

CONTROL-FLOW AND STOCHASTIC ROUTING

Early simulation-discovery approaches combined a discovered process model with stochastic information from observed executions [?]. The process model defines which activities may occur, while stochastic weights estimate the probability of alternative paths [? ?].

The discovered control-flow structure defines the behaviour that the simulator can generate. Too much generalisation may allow implausible paths, while a restrictive model may miss valid process variation [? ?]. The balance between fitness and precision defines the behavioural range of the simulator [?].

TEMPORAL AND RESOURCE BEHAVIOR

Early simulation-discovery approaches estimate execution times, waiting behaviour, and resource characteristics from event-log distributions [? ?]. Later approaches model calendars, resource pools, allocation rules, and differences between resources [? ?].

Timing often depends on operational context. Execution and waiting times may change with workload, resource availability, case characteristics, or earlier process behaviour. Resource selection may depend on more than a fixed activity-resource mapping [? ?]. Recent approaches model these aspects conditionally instead of using one global distribution per activity or resource.

ARRIVAL BEHAVIOR

Classical simulation-discovery approaches estimate inter-arrival times from historical case-start timestamps [? ?]. Later approaches condition arrivals on time or context. This allows the simulator to model demand that changes across hours, weekdays, or operating periods instead of using one pooled arrival process [?].

2.5.3 DATA-AWARE AND PROCESS-STATE-AWARE SIMULATION

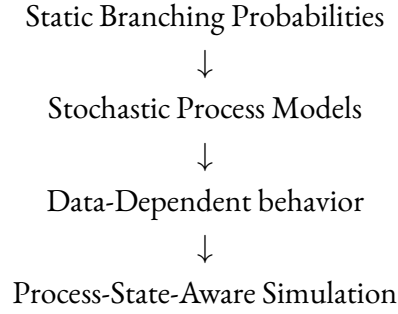
Classical stochastic simulation models often estimate one set of routing probabilities for all cases. For example, if two paths occur with frequencies of 70 % and 30 %, the model may use these probabilities for every case. This describes how often each path occurred, but not why cases followed different paths [? ?].

Data-aware simulation conditions stochastic behaviour on process data. Mannhardt et al. [?] extend stochastic Petri nets with transition weights that depend on process variables. The model can then capture differences between types of cases.

De Leoni et al. [?] extend this idea through the concept of the *process state*. Rather than conditioning decisions only on static case attributes, the process state may also describe information accumulated during the execution of the case, including process-variable values and the

history of previously executed activities. Their results show that incorporating this richer representation of the current process state can improve the estimation of activity-firing probabilities compared with basic frequency-based branching probabilities.

The development can be summarised as:



Recent work extends data awareness beyond routing behavior. López-Pintado et al. [?] investigate the discovery and simulation of data-aware business processes more broadly, reflecting a shift toward simulation models in which multiple behavioral dimensions depend on process context rather than only on marginal historical frequencies.

2.5.4 PROSiT AS A PROBABILISTIC DATA-AWARE SIMULATION FRAMEWORK

ProSiT applies data awareness to several simulation dimensions [?]. Its probabilistic white-box predictors model control-flow decisions, execution times, waiting times, resource selection, and arrival behaviour.

The framework uses probabilistic decision trees to split the feature space. Each leaf contains a local probability distribution. Simulation behaviour can vary by context while remaining stochastic. This differs from both a model with one global distribution per parameter and a deterministic model that returns one value.

ProSiT can use trace attributes, execution history, resource information, temporal information, and workload features [?]. It does not support every event-level attribute that changes during execution. The process-state approach of Mannhardt et al. [?] and de Leoni et al. [?] covers a broader range of context than the current ProSiT implementation.

ProSiT is useful for this thesis because its level of contextual conditioning can be changed. This allows a comparison of model expressiveness and held-out fidelity in the CTB truck process.

CONFIGURABLE EXPRESSIVENESS IN PROSiT. Two ProSiT configuration parameters are especially relevant for the experimental design in Section 4.5 [?]:

- `max_depth_tree` controls the maximum depth of the probabilistic decision trees used for conditional simulation behavior. Setting `max_depth_tree=0` disables conditional tree-based modelling and yields an unconditional distribution-based configuration. Increasing the depth allows the model to distinguish more detailed combinations of contextual features, while also increasing model complexity and potential overfitting.
- `use_workload_features` determines whether workload-related features are made available to the waiting-time and resource-selection models. When enabled, these models can condition their behavior on the current operational workload and queue state in addition to the remaining contextual attributes.

These parameters define the later comparison between a distribution-based configuration, a data-aware configuration, and a data-aware configuration with workload information. The evaluation tests whether the added model detail improves simulation fidelity.

2.6 EXPLAINABLE AND WHITE-BOX SIMULATION

2.6.1 EXPLAINABILITY IN SIMULATION-BASED DECISION SUPPORT

Business Process Simulation supports decisions about resources, process design, capacity, and investments. Decision makers need to understand why a model produces an outcome and how that outcome changes when parameters are modified. Traditional BPS models are transparent because their control flow, routing probabilities, resources, and timing assumptions are defined by hand [?].

Machine-learning models can learn behaviour from historical event data and may improve prediction [?]. However, their internal logic can be difficult to inspect. A model may reproduce historical behaviour without showing why it selected a route, waiting time, or resource. This makes validation and what-if analysis harder [?]. For what-if analysis, users must be able to change parameters or rules and judge whether the model response is reasonable. Explainable simulation tries to combine data-driven prediction with visible decision logic.

2.6.2 PROBABILISTIC WHITE-BOX SIMULATION MODELS

White-box simulation makes learned decisions visible and editable. The RIMS framework combines a discrete-event simulation with predictive models at runtime while keeping an inspectable simulation model [?].

Vinci et al. introduced ProSiT (*PROcess Simulation Tool*) as a white-box simulation-discovery framework [?]. ProSiT uses probabilistic decision trees for several parts of the simulation:

- control-flow decisions,
- activity execution times,
- waiting times,
- resource assignments, and
- arrival processes.

The tree conditions are visible, and each leaf contains a probability distribution. Users can inspect and change these rules without retraining a deep-learning model. The distributions keep the stochastic variation needed for simulation [?].

Published evaluations show that probabilistic white-box models can reach similar accuracy to black-box alternatives while being easier to inspect and configure [?]. However, the evidence mainly comes from benchmarks and business-process settings. There is little evidence from physical logistics systems such as container terminals. Explainability is difficult to measure. A decision tree is easier to read than a neural network, but this does not prove that its rules are correct or useful. White-box simulation must balance accuracy, explainability, and configurability.

For terminological precision, the relationships exposed by such learned rules are statistical associations conditional on the training data and model specification. Their visibility supports inspection and deliberate configuration, but does not by itself establish causal effects.

2.7 SIMULATION VERIFICATION, VALIDATION, AND TRUST-WORTHINESS

Validation asks whether a simulator is accurate enough for its intended use, not only whether it runs. A model may reproduce historical truck turnaround distributions but still fail to predict an intervention outside the range of the observed data [? ?]. Automatically discovered simulation models do not remove this requirement. They replace some manual assumptions with assumptions about event-log semantics, discovery algorithms, data attributes and fitted distributions, all of which require examination.

2.7.1 DISTINCTION BETWEEN VERIFICATION AND VALIDATION

Sargent separates several questions that are often conflated under the single term *validation* [?]. *Conceptual-model validity* asks whether the theories, assumptions and logical relationships in the conceptual model are reasonable for the intended purpose. *Computerized-model verification* asks whether that conceptual model has been implemented correctly. *Data validity* concerns whether the data used to construct, calibrate and evaluate the model are adequate. Finally, *operational validity* asks whether the computerised model’s output behaviour is accurate enough over the domain in which the model is intended to be used. Historical-data validation—running the model under historical input conditions and comparing its output with observed system data, is one technique for assessing operational validity [?].

Table 2.7 maps these distinctions to the type of evidence needed in an automatically discovered BPS study. The categories work complementary, as for example a low event-log distance is evidence of operational fidelity, but it cannot show that a timestamp proxy has the correct physical meaning or that the simulator implemented a resource constraint as intended.

This distinction clarifies the scope of this thesis. The real-versus-simulated event-log comparison is a form of held-out historical-data or operational validation. The event-log construction and the physical RMG capacity constraint contribute data- and conceptual-validity evidence, while executable case contracts and provenance checks contribute verification evidence. None of these alone proves that every what-if response is the true causal response of the terminal.

Question	Validity concept	Typical evidence
Is the abstraction reasonable?	Conceptual-model validity	Domain review of case boundaries, event semantics, resource meanings and physical constraints
Was the abstraction implemented correctly?	Computerized-model verification	Unit and integration tests, structural contracts, reproducible seeds and parameter-provenance checks
Are discovery and evaluation data suitable?	Data validity	Event-log quality checks, timestamp semantics, leakage prevention and a training/test separation
Does the model reproduce observed behaviour?	Operational validity	Comparison of simulated and held-out real logs across multiple process perspectives and stochastic replications
Does a scenario predict the true intervention effect?	Predictive validity outside the historical regime	Intervention observations, controlled field evidence or additional domain-supported sensitivity analysis

Table 2.7: Validation questions and corresponding evidence in an event-log-discovered simulation study, based on Sargent's framework [?].

2.7.2 FROM HISTORICAL LOGS TO OPERATIONAL VALIDATION

Process Mining provides a natural representation for historical-data validation because both the observed system and the simulator can be represented by event logs. Rozinat et al. established an early version of this principle: they discover control-flow, data, performance and resource perspectives, integrate them into a coloured Petri-net simulation model, simulate a second event log, and analyse how well that generated log approximates the characteristics of the original log [?]. This is stronger than checking whether the discovered model can execute; it evaluates whether its generated behaviour resembles recorded behaviour.

Camargo et al. formalise the same idea as an optimisation objective. Candidate BPS configurations generate event logs and hyperparameter optimisation searches for the configuration that maximises similarity to observed log behaviour [?]. Their later comparison of data-driven simulation and deep generative models further shows why quality must not be inferred from trace frequencies alone: conventional data-driven simulators may reproduce activity sequences and their frequencies while performing worse on temporal dynamics [?].

The implication is that operational fidelity is multi-dimensional. Relevant perspectives include control flow, activity and case timing, case arrivals, congestion and resource behaviour. A model can be close to the real log in one perspective and far away in another; collapsing all discrepancies into one number can hide the mechanism that is incorrectly represented [? ?].

2.7.3 MULTI-PERSPECTIVE EVENT-LOG COMPARISON

Chapela-Campa et al. provide a validation framework that matches the evaluation problem of this thesis [? ?]. Their starting point is that the true BPS model is normally unavailable and that different simulation engines use incomparable internal formats. They evaluate a model indirectly as a generator: multiple logs generated by the BPS model are compared with an actual event log that was not used to construct the model. The framework separates control-flow, temporal and congestion perspectives and reports a measure for each instead of using one aggregate score for overall model quality. The measures are evaluated using controlled perturbations and by comparing alternative automatically discovered BPS models, showing that perspective-specific measures can identify the source of a discrepancy [?].

CONTROL-FLOW PERSPECTIVE

Control-flow output quality asks whether the simulator generates activity sequences and sequence frequencies resembling the observed process. It is distinct from replay fitness of a control-flow model: fitness asks whether real traces are allowed by the model, whereas output validation asks which traces the stochastic simulator actually generates and with what frequency.

Stochastic conformance work applies transport distances to probability distributions over traces or other behavioural abstractions [?]. Chapela-Campa et al. instead define two simulation-oriented log distances [? ?]. Control-Flow Log Distance (CFLD) computes the normalised Damerau–Levenshtein distance between trace pairs and uses the Hungarian algorithm to find the minimum-cost one-to-one matching between the two logs. CFLD compares complete traces; it is not merely a comparison of variant-frequency vectors. Because this matching is computationally expensive, N-Gram Distance (NGD) compares normalised frequencies of local activity subsequences. NGD is scalable and detects local ordering differences, although it loses some complete-trace information.

This thesis does not claim to implement the full Chapela-Campa metric suite. Its structural evidence combines held-out Petri-net conformance, explicit case-language contracts and yard-activity frequency error. These measures answer the CTB-specific structural questions, while the Chapela-Campa framework supplies the rationale for keeping structural evidence separate from temporal fidelity.

TEMPORAL PERSPECTIVE

Identical or similar traces do not imply similar process performance. Temporal validation considers whether the simulator reproduces when events occur, how cases progress after arrival and how long activities and complete cases take. Camargo et al. show empirically that a generative model may reproduce control-flow frequencies but still miss temporal dynamics [?].

The Chapela-Campa framework distinguishes three event-distribution measures [? ?]. *Absolute Event Distribution* (AED) compares event counts across absolute calendar-time bins; *Circadian Event Distribution* (CED) compares recurring weekday and hour-of-day patterns; and *Relative Event Distribution* (RED) offsets timestamps by each case’s arrival and compares its temporal progression. AED and RED mean *event distribution*, not absolute or relative error distance. For end-to-end performance, Cycle Time Distribution (CTD) compares the empirical distributions of complete case durations.

The CTB evaluation uses more directly interpretable distributions at the level of its research questions: inter-arrival time, per-activity yard service time, model-derived pre-service delay and case turnaround time. Reporting complete distributions rather than only their means is useful because equal averages can conceal differences in dispersion, skewness or tails. Per-activity results prevent common RMG events from concealing weaker fidelity for less frequent yard activities.

ARRIVAL AND CONGESTION PERSPECTIVES

Congestion is a system-level consequence of the interaction between demand, service requirements and resource availability. It cannot be validated from activity-duration marginals alone. Chapela-Campa et al. use Case Arrival Rate (CAR) to compare binned arrival counts and shapes, and CTD to compare another congestion component, the length of time cases remain in the system [? ?]. An arrival model can be temporally unrealistic even if it generates the correct total number of cases.

This issue matters at CTB: truck demand determines competition for yard resources, and simultaneous arrivals result from the minute-resolved source data. The evaluation reports inter-arrival distance, simultaneous-arrival share and arrivals per elapsed hour alongside turnaround and yard-service distributions. These measures do not directly observe a physical queue; they assess whether the generated workload and its resulting temporal behaviour match what the available event log can identify.

WASSERSTEIN DISTANCE

For two one-dimensional distributions F and G with finite first moments, the first Wasserstein distance can be written as

$$W_1(F, G) = \int_{-\infty}^{\infty} |F(x) - G(x)| dx. \quad (2.3)$$

It has the optimal-transport interpretation of the minimum cost of moving probability mass from one distribution to the other. For durations measured in minutes, W_1 is expressed in minutes, which gives the discrepancy an operational scale.

Earth Mover's Distance (EMD) is related, but it is not always the same measure. In the formulation used by Chapela-Campa et al., EMD may compare time series with unequal total mass and penalise the creation of missing mass, whereas first Wasserstein distance assumes equal nor-

malised mass [? ?]. For empirical probability distributions with unit mass, the formulations coincide. In Process Mining, related transport distances have been used for stochastic conformance checking over behavioural distributions [?]. In this thesis, the term *EMD* for service, inter-arrival and turnaround samples denotes the one-dimensional W_1 distance between empirical distributions.

KOLMOGOROV–SMIRNOV STATISTIC

The two-sample Kolmogorov–Smirnov (KS) statistic is the maximum vertical difference between the empirical cumulative distribution functions of the simulated and observed samples [?]:

$$D_{n,m} = \sup_x |F_{n,\text{sim}}(x) - F_{m,\text{real}}(x)|. \quad (2.4)$$

Unlike W_1 , it is dimensionless and bounded by zero and one. The two statistics are complementary: W_1 measures integrated displacement on the operational scale, while KS reports the largest local CDF gap. The KS statistic can be accompanied by a significance-test p-value, but that p-value answers a different question from practical fidelity. With large event logs, small and operationally unimportant differences can be statistically significant. The magnitude of the statistic and the distributional plots must be interpreted separately from binary significance decisions.

ADDITIONAL REQUIREMENTS FOR DATA-AWARE DISCOVERY

A data-aware simulator conditions behaviour on case attributes, process state or workload information rather than using only global distributions and branching frequencies [? ? ?]. This adds a validation layer. First, every predictor used at a simulated decision point must be available at that point; attributes computed from a later outcome create target leakage even if aggregate log similarity is high. Second, discovery and model selection must be separated from the final test log. Third, the generated log must satisfy process and domain invariants that aggregate distances may overlook.

White-box rules help because their split conditions and leaf distributions can be inspected and modified [?]. Interpretability, however, is not equivalent to validity or causality. A readable rule may encode a spurious association, and two differently specified models may reproduce the same marginal distribution. Validation should combine held-out output fidelity with rule inspection, leakage checks, structural contracts, domain constraints and sensitivity analysis. This matters in the CTB application because timestamp fields are operational proxies and a resource

label does not always denote a physically interchangeable crane or storage position.

STOCHASTIC REPLICATION AND CONFIDENCE INTERVALS

A BPS model is a stochastic generator. One fixed model can generate many possible event logs, whereas the real test log is only one observed realisation. A single simulated log mixes model fidelity with the random draws of that run. Chapela-Campa et al. address this problem directly: they generate K logs from K simulation runs, match their case count and start point to an actual test log that was not used for model construction, compare every generated log with that test log, and construct confidence intervals for each perspective-specific distance [?]. Their controlled evaluation uses $K = 10$ replications and reports means and 95 % intervals.

This design is consistent with classical simulation-output analysis, where independent replications with different random-number streams provide observations for estimating a mean and its Monte-Carlo uncertainty [?]. For replication-level values $x_1, \dots, x_N$, a two-sided Student-t interval is

$$\bar{x} \pm t_{0.975, N-1} \frac{s}{\sqrt{N}}, \quad (2.5)$$

where s is the sample standard deviation across replications.

The interpretation of this interval must remain narrow. Conditional on one frozen discovered parameter bundle and one fixed real test log, it quantifies the variability caused by simulation sampling. It does not include event-log sampling uncertainty, input-distribution estimation uncertainty, alternative discovery outcomes, measurement error or uncertainty about the correctness of a what-if response. Those require additional resampling, repeated discovery, sensitivity experiments or external domain evidence. Chapter 4 uses this conditional Monte-Carlo interpretation for its ten-seed protocol.

2.7.4 TRUSTWORTHINESS AND THE BOUNDARY OF HISTORICAL VALIDATION

Historical fidelity is necessary for a simulator that claims to represent an existing process, but it is not sufficient for unrestricted decision support. Chapela-Campa et al. identify this limit: comparison with an original event log assesses replication of the *as-is* process, not the accuracy of predictions after a process change [? ?]. Similarly, Sargent’s purpose-relative view implies that evidence collected in one operating region cannot automatically validate use in another [?]

].

Trustworthiness cannot be measured with one metric. Evidence is needed for data and conceptual validity, implementation, held-out fidelity, reproducibility, assumptions, stochastic robustness, domain review, and the limits of extrapolation. ProSiT provides probabilistic rules that can be inspected and configured [?]. Applied work shows how a discovered simulation model can be connected to a resource-allocation problem [?]. These properties support validation and scenario analysis, but they do not replace evidence for each intervention.

This thesis does not claim to solve the general trustworthiness problem of simulation discovery. It contributes a container-terminal case study: the model is tested against an unseen historical period, across structural, temporal and arrival perspectives, with repeated stochastic runs and explicit domain constraints. Scenario results are interpreted as conditional consequences of the documented model assumptions, not as experimentally established causal effects.

2.7.5 RESEARCH GAP

Container terminals are dynamic systems with limited resources and connected truck, yard, storage, and equipment processes. Simulation is used to support decisions in this setting [? ? ?]. However, most terminal simulation models are built by hand. Their construction, calibration, and maintenance require time and domain knowledge [?].

Simulation discovery creates models from event logs. Recent work covers simulation parameters, data-aware decision logic, and automated model building [? ? ?]. Data-aware approaches can use context and process state instead of only fixed branching probabilities [? ?]. ProSiT adds visible probabilistic decision models [?].

Most evaluations come from healthcare, administration, and standard business processes. Evidence from large container terminals is limited. Open questions include:

- the robustness of automatically discovered models,
- their use for operational decision support,
- their ability to reproduce congestion and workload, and
- the value of white-box simulation in logistics.

This thesis studies these questions for truck handling at HHLA Container Terminal Burchardkai (CTB). It provides evidence about the strengths and limits of simulation discovery in this setting.

3

Case Study and Data Engineering

This chapter describes the operational context and the pipeline that turns raw operational records into a simulation-ready event log. It covers the data engineering but does not yet fit models, evaluate results, or choose experimental parameters. The resulting event log is the input for Chapters 4 and 5.

Section 3.1 introduces HHLA Container Terminal Burchardkai (CTB), the truck-handling operations and the data sources. Section 3.2 describes how the pipeline reconstructs lifecycle timestamps and derives the case-level context required by ProSiT [?]. The final section describes the event log and its characteristics used for the train/test split in Section 4.3.

3.1 CASE STUDY ENVIRONMENT

3.1.1 HHLA CONTAINER TERMINAL BURCHARDKAI

This case study concerns the HHLA Container Terminal Burchardkai (CTB) in the Port of Hamburg. CTB handles approximately 2.6 million TEU (Twenty-foot Equivalent Units) annually across 315 hectares [?].

Truck process. External trucks visit CTB to deliver or receive containers. At *Gate In*, the truck

is registered at the entry gate. It then drives to the assigned yard position, where an RMG or other handling equipment loads or unloads the container. After all planned operations are completed, the truck leaves through *Gate Out*.

The CTB environment in this project has several resource constraints. Located between the waterside and the landside, 22 RMG storage blocks (T₀₆–T₂₇) serve both sides. Operationally, at most three automated gantry cranes can work in parallel at one block. The CTB has two specialised areas: an empty-container depot (called “MT” or “LL”, from the German *Leerlager*) and the manually operated HO₂ yard area. The terminal gate consists of 16 lanes, split between incoming and outgoing trucks. The number of operated gate lanes varies by day, shift and terminal resource plan.

3.1.2 DATA SOURCES

The data come from several systems used in daily terminal operations. These systems were developed over time and record different parts of the process. A Terminal Operating System coordinates them. Table 3.1 summarises the four sources used in this thesis.

Data source	Data provided
ITS schedule	Truck-visit schedules and timestamps for gate entry, handling readiness, order completion and gate exit.
Report system	Stage-level execution data, gate-entry records, exchange-area assignments, container transactions and carrier identifiers.
Yard management system	Four-hour snapshots of storage-block capacity, occupancy and utilisation.
Container master data	Container size, type, loading status, hazardous and reefer indicators, dwell time and inbound/outbound carriers.

Table 3.1: Operational data sources used for event-log engineering.

These systems generate independent data pools with different identifier schemes, requiring a data-integration procedure described in Section 3.2. Historical data from March and April 2026 (approximately eight weeks) were extracted. The raw extract covers approximately 104,000 truck visits, 113,466 container move events and 336 yard-state snapshots (8 weeks $\times$ 7 days $\times$ 6 snapshots per day).

3.2 EVENT LOG ENGINEERING

Event-log engineering is the first stage of the experimental pipeline. The objective is to construct an integrated, process-oriented event log from the raw operational data that satisfies the requirements of both process mining and data-aware simulation while retaining as much operational context as possible. This section describes the pipeline stage by stage.

Figure 3.1 summarises how the four operational data sources are transformed into the final simulation-ready event log.

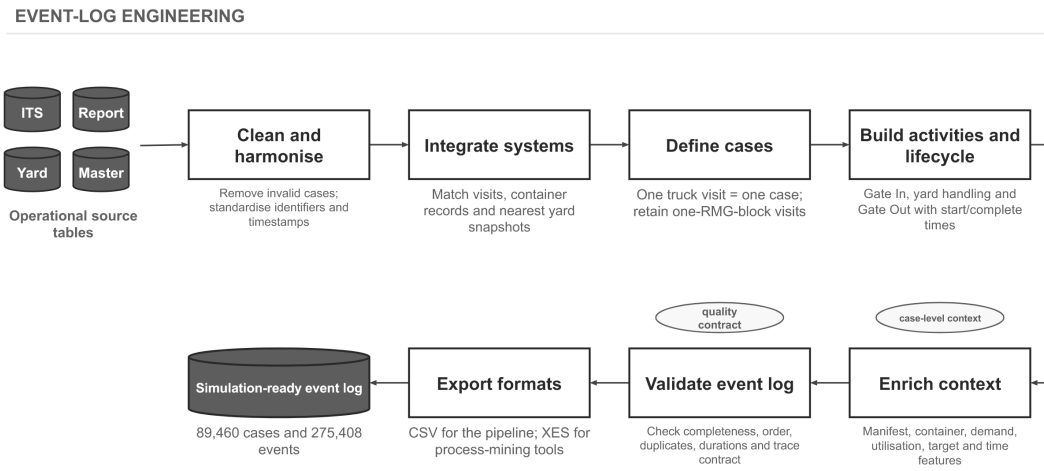


Figure 3.1: Event-log engineering pipeline. The ITS schedule, report-system records, yard snapshots and container master data are cleaned and integrated. Truck visits are then represented as cases with lifecycle events and case-level context. The final checks produce the CSV and XES event logs used for discovery and evaluation.

3.2.1 RAW DATA SOURCES AND PREPARATION

ITS SCHEDULE DATA

The ITS scheduling system provides the scheduling data for the truck operations reconstructed in this thesis. Each record represents one activity instance within a truck visit. The attributes used by the pipeline are listed in Table 3.2.

Attribute	Description
FAHRPLAN_UID	Unique truck-visit identifier used as the case ID.
LKW_KENNZEICHEN	Truck licence plate number.
ANLAUFUNKT_ HALTESTELLE	Exchange-area identifier, such as T2o, LL or VC.
FP_ZEITPUNKT_ERSTELLUNG	Time at which the schedule entry was created; used as the Gate In timestamp.
ALP_ZEITPUNKT_ BEREITMELDUNG	Time at which the handling activity was reported ready for execution.
ALP_ZEITPUNKT_ ERFUELLUNG	Time at which the handling activity was completed.
FP_ZEITPUNKT_GATEOUT	Time at which the truck exited the terminal.

Table 3.2: ITS schedule attributes used in the event-log pipeline.

All timestamp fields are parsed into standardised `datetime` objects using `pandas` with the format `%d.%m.%Y %H:%M`. Invalid or missing timestamps are flagged for removal during the cleaning phase.

REPORT SYSTEM DATA

The report system provides complementary stage-level execution data, completing the details for gate-entry events, and it links each container number to the specific truck visit at a yard handling point. The attributes used from this source are shown in Table 3.3.

A container is considered “picked up” or “delivered” by a truck if the corresponding inbound or outbound carrier identifier resolves to a truck. The report system uses detailed handover-lane codes for the exchange area; these must be mapped to the higher-level operational zones used by the scheduling system. All exchange areas beginning with “HP” (*Hochpalette*) are mapped to “LL” (*Leerlager*) to align with the schedule system’s terminology. The mapping is implemented in `002_Prep_area_mapping_report.py`.

CONTAINER MASTER DATA

Container-level attributes are extracted from the container master data and linked to events through the matched `container_id`. Table 3.4 describes the retained fields and derived measures.

The complexity score is derived during the container-master-data preprocessing stage and com-

Attribute	Description
TruckVisitGkey	Truck-visit identifier used for cross-system matching.
TruckLicenseNbr	Truck licence plate number.
CallupTime	Time at which the truck physically entered the terminal.
StageStartTime	Start time recorded for the reported process stage.
StageId	Stage identifier, such as OCR, Gate-In, yard or Exit.
TruckExchangeArea	Yard handling point visited for a specific container.
TranCtrNbr	Container number associated with the transaction.
UnitInboundCarrierId	Inbound carrier identifier, recorded as a truck plate, vessel voyage number or train ID.
UnitOutboundCarrierId	Outbound carrier identifier, recorded as a truck plate, vessel voyage number or train ID.

Table 3.3: Report-system attributes used in the event-log pipeline.

Attribute	Description
Container size	TEU-equivalent size category (20, 40 or 45 feet).
Full/empty status	Binary loading-status indicator (Leer/Voll).
Reefer status	Indicator for refrigerated containers, derived from the ISO code.
Hazardous-material status	Indicator derived from the IMCO code.
Container type	Operational type, such as standard, open-top, platform or bulk.
Dwell time	Hours already spent in the terminal for a receive operation.
Container complexity score	Composite measure based on size, loading status, reefer and hazardous-goods requirements, container type and dwell time.

Table 3.4: Container-master attributes used for context enrichment.

bines container type, loading status, hazardous-goods indicators, reefer requirements and other handling-related properties into a single numerical measure. Higher values indicate visits expected to require more complex handling.

YARD STATE DATA

Yard state data provides time-series information on storage-block capacities and utilisation rates. Each record contains ZEITPUNKT (observation timestamp), AREA (storage area identifier), KAP_TEU_THEORETISCH (theoretical TEU capacity), BEL_TEU (current TEU occupancy) and AUSLASTUNG_% (utilisation percentage). Because observations are irregularly spaced, the data is resampled to a uniform one-minute frequency using linear interpolation on the numeric fields (script 004_Prep_create_interpolate).

TRUCK DEMAND DATA

Truck demand information is derived from the scheduling system by aggregating truck arrivals over rolling time windows. For each exchange area and one-minute time interval, the feature TRUCK_DEMAND_LAST_1H is calculated as the number of trucks that became ready for processing in that area during the preceding sixty minutes. The demand pipeline (script 005_Prep_create_truck_demand) first resamples the schedule data to one-minute intervals based on ALP_ZEITPUNKT_BEREITMELDUNG. It then applies a rolling 60-minute window and counts truck arrivals separately for the Gate, the individual RMG blocks To6–T27, VC, LL and MT. The block-level RMG series are also aggregated into one yard-section demand measure.

The resulting demand features capture workload pressure at the time of case arrival. They are included as contextual predictors because such pressure may influence waiting times and resource-allocation decisions; their empirical contribution is evaluated under RQ3 in Section 4.5.

3.2.2 DATA CLEANING AND INTEGRATION

CROSS-SYSTEM IDENTIFIER MATCHING

The scheduling and report systems use different identifiers for truck visits. To create a unified event log they are linked through composite keys. Truck visits are matched by (TruckLicenseNbr, CallupTime) (LKW_KENNZEICHEN, FP_ZEITPUNKT_ERSTELLUNG) using a left join that preserves all schedule records and augments them with the report-system TruckVisitGkey when available. Container numbers are then matched via (TruckVisitGkey, TruckExchangeArea) $\equiv$ (TruckVisitGkey, ANLAUF). When the first matching step fails, a fallback is applied to single-exchange-area visits: if a truck

visit contains only one yard activity and no container identifier was matched, the container ID is inferred from any other transaction associated with the same `TruckVisitGkey`. The integration logic is implemented in `01_match_container_ids_to_schedule.py`.

INVALID PROCESS INSTANCES

The cleaning pipeline removes complete cases rather than individual records: whenever a truck visit violates one of the defined quality criteria the entire process instance is excluded, so that only internally consistent process executions are retained. The cleaning procedures are implemented in `00_its_fahrplan_raw_to_fahrplan_cleaned_0304.py`.

MISSING LIFECYCLE INFORMATION. Cases are removed if one or more required lifecycle timestamps are missing: `FP_ZEITPUNKT_ERSTELLUNG`, `ALP_ZEITPUNKT_BEREITMELDUNG` or `ALP_ZEITPUNKT_ERST`. Such records cannot be transformed into valid lifecycle events. The quality report identified 1,267 affected cases ($\approx 1.34\%$ of all truck visits).

TEMPORAL INCONSISTENCIES. Cases are removed when temporal constraints are violated. First, activities with negative execution durations (`ALP_ZEITPUNKT_ERFUELLUNG < ALP_ZEITPUNKT_BEREITMELDUNG`) indicate timestamp errors and are excluded. Second, overlapping activities within a single visit ($\text{start}_{i+1} < \text{complete}_i$ after sorting) contradict the operational logic (a truck can only be at one handling location at a time). The cleaning procedure identified three cases with negative durations and 43 with overlapping activities.

EXTREME RUNTIME OUTLIERS. Two further runtime filters are applied. The first removes cases with excessive gate-out delay ($t_{\text{GateOut}} - t_{\text{LastCompletion}} > 10\text{h}$); such records commonly indicate system failure. The second removes visits whose total turnaround time exceeds twelve hours ($t_{\text{GateOut}} - t_{\text{GateIn}} > 12\text{h}$), which is operationally implausible. The filters identified 22 extreme gate-out cases and 22 twelve-hour outliers.

PROCESS COMPLETENESS FILTERS

CONTAINER IDENTIFICATION COVERAGE. Overall, 111,646 of 113,136 operational records could be assigned a valid `TruckVisitGkey` (98.68%), and 110,866 records could be assigned a valid container identifier (97.99%). Despite the fallback procedure, 2,270 records belonging to 2,034 truck visits remained without a valid container assignment. Since container-related attributes are needed for the later context enrichment and simulation-discovery stages, these process in-

stances are removed entirely (a complete-case filter). This removes 2,034 truck visits and 3,397 associated records, leaving 91,335 complete truck visits and 109,739 operational records.

UNKNOWN ACTIVITY TYPES. During activity construction, cases containing activities classified as *unknown* are removed. Such events cannot be mapped to operational process steps with enough confidence. They would introduce ambiguity into process discovery and simulation discovery.

PROCESS SCOPE RESTRICTION. The event log is restricted to truck visits containing exactly one interaction with an automated RMG block. Visits may still contain additional activities in other yard areas (empty-container yard or VC yard), but only a single RMG block interaction is permitted. This restriction ensures that RMG-specific workload, utilisation and demand measures can be assigned one value per case. It forms the basis for the block-focused what-if experiments in Section 4.8.

EVENT-LEVEL CLEANUP. Immediately before exporting the event log, remaining events with an *_unknown* suffix are removed. Unlike the previous filters, this is an event-level cleanup rather than a case-level exclusion; its purpose is to remove residual events that cannot be interpreted within the final process model.

3.2.3 CASE DEFINITION

A *case* in the event log corresponds to a single truck visit, uniquely identified by `FAHRPLAN_UID`. Each case represents the complete sequence of activities executed by one truck between gate entry and gate exit.

The data-aware simulation framework used in this thesis [?] accepts only case-level contextual attributes. To attach block-specific context features (`target_utilization`, `target_demand`, `target_rank`) consistently, the design decision was made to include only process instances that interact with exactly one RMG block. Visits involving multiple RMG blocks are excluded because a single value could not be assigned to their block-specific congestion measures. Visits to other yard areas (VC or LL) that involve exactly one RMG block are retained. The filtering logic is implemented in `filter_one_rmg_only.py`.

3.2.4 ACTIVITY CONSTRUCTION

Activities are defined by combining an *operation type* (receive, delivery or mixed) with a *resource type* (an RMG block, LL or HO2). Table 3.5 lists the resulting labels.

Activity	Definition
Gate In	Truck registration at entry gate
RMG_receive	RMG unloads container from truck (import)
RMG_delivery	RMG loads container onto truck (export)
RMG_mixed	Both receive and delivery at same block
LL_receive	Empty-container depot: unload
LL_delivery	Empty-container depot: load
LL_mixed	Empty-container depot: unload and load
HO2_receive	Manual area: unload
HO2_delivery	Manual area: load
HO2_mixed	Manual area: unload and load
Gate Out	Truck departure

Table 3.5: Activity labels used in the final event log.

The 22 individual RMG blocks (To6–T27) are retained as distinct resources to preserve block-specific utilisation patterns.

3.2.5 ACTIVITY TIMESTAMPS AND TEMPORAL SEMANTICS

The CTB operational systems provide timestamps from which two observed lifecycle states are constructed for each activity. They are summarised in Table 3.6.

Attribute	Description
start:timestamp	Observed or reconstructed time at which execution of the activity began.
time:timestamp	Observed or reconstructed time at which execution of the activity finished.

Table 3.6: Observed lifecycle timestamp attributes in the final event log.

The *service time* (or execution time) is the directly identifiable temporal quantity:

$$ET_i = t_{\text{complete},i} - t_{\text{start},i}. \quad (3.1)$$

An explicit `enabled:timestamp` is not reconstructed in the input event log. Consistent with ProSiT [?] and earlier simulation-discovery work [? ?], activity enablement is obtained from process-model alignment during the simulation-parameter discovery phase (Section 4.5). ProSiT uses an A*-based optimal alignment as defined in Section 2.3.1 to relate each observed event to a transition and determine when that transition becomes enabled in the discovered Petri net, then computes a model-derived pre-service delay relative to both the enablement time and the resource’s availability schedule.

TIMESTAMP MAPPING BY ACTIVITY TYPE

Table 3.7 shows how the lifecycle timestamps are mapped for each activity type.

Activity type	Start timestamp	Complete timestamp
Gate In	FP_ZEITPUNKT_ERSTELLUNG	Same value as the start timestamp.
Yard activity	ALP_ZEITPUNKT_ BEREITMELDUNG	ALP_ZEITPUNKT_ ERFUELLUNG
Gate Out	FP_ZEITPUNKT_GATEOUT	FP_ZEITPUNKT_GATEOUT

Table 3.7: Lifecycle timestamp mapping by activity type.

The OCR gate timestamp (`StageStartTime`) from the report system is retained in the auxiliary column `ocr_timestamp` for diagnostic purposes. It is not mapped to activity enablement.

MODEL-DERIVED PRE-SERVICE DELAY

Because activity enablement is derived from the process model, the resulting quantity

$$D_{\text{pre-service},i} = t_{\text{start},i} - t_{\text{enabled,model},i} \quad (3.2)$$

is a *model-derived pre-service delay*. In ProSiT terminology this is called “waiting time,” but in the CTB application it must not be interpreted as pure queueing or resource-induced waiting. Physical truck movement between consecutive terminal locations is not represented as a separate activity in the discovered process model. The model-derived interval may therefore combine physical movement between locations, internal traffic and congestion, resource availability and queue waiting, and other delays not represented by the control-flow model. This is a domain-specific limitation when transferring business-process simulation semantics to physical terminal operations and is a methodological limit of this thesis (discussed further in Section 5.9).

METHODOLOGICAL NOTE: THE ABANDONED TRANSITION-BASELINE APPROACH

An earlier version of the event-log pipeline estimated lower-tail route-specific transition durations from the scheduling data (5th percentile of the empirical distribution per transition pair, `transition_baseline.csv`) and used these to reconstruct an explicit `enabled:timestamp` in the input event log. Closer inspection revealed two issues. First, the statistical route baselines are aggregate quantiles rather than event-specific observations of physical arrival. Their values therefore depend on the selected percentile (Section 5.7). Second, the estimated readiness timestamp sometimes occurred after the observed start timestamp. This produced negative waiting times that had to be clipped to zero. ProSiT does not require an externally constructed `enabled` timestamp. It derives enablement from the aligned process model [?]. The final method keeps only the observed start and completion timestamps and leaves enablement reconstruction to the process-model alignment. This keeps the data and simulation semantics consistent.

The transition-baseline data is preserved in the repository as diagnostic material and may be useful for future work that introduces an explicit spatial-transition model.

3.2.6 CONTEXT ENRICHMENT

To enable data-aware simulation, the event log is enriched with contextual attributes describing the operational state at the time of process execution. These attributes are organised into five categories.

INFORMATION AVAILABLE AT TRUCK ENTRY

The scheduling record contains the complete planned visit manifest before the truck enters the terminal. The number of planned stops, the numbers of delivery and receive transactions, the planned target area and the resulting final visit-complexity score are known at Gate In. The pipeline reproduces this entry-time information by aggregating all rows of the same `FAHRPLAN_UID`; it does not infer these attributes from later realised durations or completion outcomes. `visit_complexity` is a deterministic summary of the planned stop/container manifest (visit size, container-handling complexity and mixed-flow complexity), not an ex-post performance measure. Using these variables as case attributes does not leak the simulated future provided that the same planned manifest is supplied for a prospective run.

The implementation detail that the target block is reconstructed from the planned stop's area-specific activity label can otherwise look like look-ahead: after genericising T06--T27 activities,

the reconstructed block is copied into `primary_target_area`. Semantically, however, the source is the planned `ANLAUFPUNKT_HALTESTELLE` attached to the visit order and available at entry, not the later activity-completion timestamp. This distinction defines the deployment contract for all entry-known case attributes.

CONTAINER FEATURES

Container-related context is aggregated to case level so that visit properties can serve as case-level attributes in ProSiT (script `03_aggregate_container_details_case_level.py`). The resulting `n_containers` attribute counts the unique containers handled during the visit. The binary attributes `has_hazardous` and `has_reefer` are aggregated using a logical OR across the complete manifest. The `full_ratio` records the share of loaded containers, while `container_complexity_mean` and `container_complexity_sum` describe the average and cumulative handling complexity of the visit.

YARD FEATURES

Yard state features capture the storage capacity and utilisation status of different terminal areas at truck arrival (Gate In event). These features are matched temporally using `pd.merge_asof` with a nearest strategy. The joined snapshot provides the overall `gate_utilization`, the aggregated `rmg_utilization`, and the area-level `vc_utilization` and `mt_utilization`. For single-block visits, `target_utilization` gives the utilisation of the assigned RMG block.

These features are added in `06_add_utils_to_eventlog.py`.

DEMAND FEATURES

Demand features quantify operational load pressure at different terminal areas over a rolling 60-minute window (scripts `05_add_demand_features_to_eventlog.py` and `08_add_all_target_features`). The general `gate_demand` counts trucks processed across the terminal during the preceding hour. The `rmg_demand`, `vc_demand` and `mt_demand` attributes separate this load by operational area. For single-block visits, `target_demand` records the demand for the assigned RMG block.

TARGET BLOCK FEATURES

For single-block truck visits, the assigned target RMG block is required for the context model. Beyond the target-specific utilisation and demand, `target_rank` gives its ordinal position among the RMG blocks based on utilisation at case arrival. The corresponding `target_rank_group`

reduces this rank to `top_3`, `mid` or `low`. The attributes `primary_target_area` and `target_area` retain the assigned block identifier.

These features are reconstructed from the planned, area-specific stop labels before the RMG activities are genericised and are then carried as case attributes (scripts `07_convert_to_RMG_filter_LL+H02_onl` and `08_add_all_target_features.py`). Their information time is Gate In as specified in Section 3.2.6; the later `org:resource` assignment is not used as an execution outcome to be predicted.

3.2.7 FINAL EVENT LOG STRUCTURE

The final event log combines process-execution data, lifecycle information and operational context within a single simulation-ready representation. Table 3.8 groups the process and case-level attributes used by the simulation.

All contextual attributes are attached at case level and remain constant throughout the case. This design is consistent with the current ProSiT feature-discovery approach, which mainly utilises case-level context information rather than event-level dynamic state [?].

3.2.8 EVENT LOG CHARACTERISTICS

After the full preprocessing pipeline the final event log contains 89,460 truck visits represented by 275,408 events. The activity distribution is shown in Table 3.9. Table 3.10 summarises the overall event log.

Every case contains a `Gate In` event, one or more operational yard activities and a final `Gate Out` event. Operational activities can occur at an RMG block, the empty area or the manual area, depending on the operational requirements of the visit.

[INSERT MOST COMMON VARIANTS?]

The generated event log is exported as the full CSV `s6_eventlog_target_rank_features.csv`. The frozen temporal partitions are exported as `s6_train.csv`, `s6_test.csv` and the corresponding `xes_files/s6_train.xes` and `xes_files/s6_test.xes`. CSV is the pipeline format; XES enables compatibility with `pm4py`, `ProM` and `ProSiT` for process discovery and conformance analysis.

The chronological character of the log is preserved, as every case follows a increasing temporal

Attribute	Description
Baseline attributes	
case:concept:name	Unique truck-visit identifier (FAHRPLAN_UID).
concept:name	Activity executed during the visit.
start:timestamp	Observed or reconstructed activity start timestamp.
time:timestamp	Observed or reconstructed activity completion timestamp.
org:resource	Operational location of the activity.
Visit-related attributes	
process_flow_type	Visit type: delivery, receive or mixed.
n_containers	Number of containers in the visit.
n_stops	Number of planned operational stops known at Gate In.
n_deliveries	Number of planned delivery transactions known at Gate In.
n_receives	Number of planned receive transactions known at Gate In.
has_hazardous	Indicator for at least one IMDG container.
has_reefer	Indicator for at least one reefer container.
full_ratio	Share of loaded containers.
visit_complexity	Composite score of the complete planned visit manifest.
Demand attributes	
gate_demand	Truck demand across the terminal during the preceding hour.
rmg_demand	Truck demand in the RMG yard during the preceding hour.
vc_demand	Truck demand in the VC area during the preceding hour.
mt_demand	Truck demand in the manual yard during the preceding hour.
Utilisation attributes	
gate_utilization	Overall yard utilisation at truck arrival.
rmg_utilization	Aggregate RMG-yard utilisation at truck arrival.
vc_utilization	VC-area utilisation at truck arrival.
mt_utilization	Manual-yard utilisation at truck arrival.
Target attributes	
target_utilization	Utilisation of the primary target area at truck arrival.
target_demand	Demand for the primary target area during the preceding hour.
target_utilization_bin	Quantile category of target-area utilisation.
target_demand_bin	Quantile category of target-area demand.
target_rank	Utilisation rank of the target among all RMG blocks.
target_rank_group	Categorised target rank: top_3, mid or low.
primary_target_area / target_area	Planned target area known at Gate In.

Table 3.8: Process and contextual attributes passed to ProSiT.

Activity	Frequency
Gate In	89,460
Gate Out	89,460
RMG_receive	39,965
RMG_delivery	22,504
LL_delivery	13,230
HO ₂ _delivery	7,224
HO ₂ _receive	5,464
RMG_mixed	4,939
LL_receive	2,073
HO ₂ _mixed	1,025
LL_mixed	64

Table 3.9: Activity distribution in the final event log.

Statistic	Value
Cases	89,460
Events	275,408
Distinct activities	11
Observation start	2026-03-02 05:27
Observation end	2026-04-30 11:54
Distinct RMG resources	22 (To6–T27)
Additional pools	LL, HO ₂ , Gate In, Gate Out

Table 3.10: Final event log summary.

behavior from Gate In to Gate Out. This property is further examined in Chapter 4 to define a temporal train/test split at case level (see Section 4.3).

4

Experiments and Evaluation Framework

This chapter describes how the simulation models are discovered, executed and evaluated. The experimental design starts from the event log constructed in Chapter 3 and ends with the historical validation and operational scenario runs reported in Chapter 5.

Section 4.1 defines the goals of the experiments. Section 4.2 describes the solution architecture and connects its processing blocks. Section 4.3 defines the held-out train/test split that separates discovery from evaluation. Section 4.4 describes the initial control-flow discovery, the domain check, the prefix-trie replacement, and its held-out validation. Section 4.5 defines the three ProSiT configurations that will be compared: distribution-based (no-rules), data-aware with decision trees (rules) and data-aware with additional queue-length features (rules+workload). Section 4.6 defines the multi-seed Monte-Carlo replication protocol that produces the confidence intervals used throughout Chapter 5. Section 4.7 defines the evaluation metrics. Section 4.8 defines the two operational what-if scenarios. Section 4.9 defines the transition-baseline sensitivity study. Section 4.10 documents the ProSiT workarounds required for the CTB event log.

4.1 GOALS OF THE EXPERIMENTS

The experiments assess whether a data-aware process simulation model can be constructed from historical CTB truck-operation data and used as a credible representation of the observed process. The aim is not to optimise the terminal or to show that a more complex model is automatically better. The aim is to test what the discovered model reproduces, where the discovery workflow requires domain intervention, and what can be learned from its responses to controlled operational changes.

The first goal is to construct an executable simulation model from the event log while preserving the physical sequence of a truck visit. The expected workflow starts with automatic process discovery. Its output is checked not only through process-mining quality measures but also against the case contract of one Gate In, sequential yard handling, and one final Gate Out. If the discovered structure violates this contract, the experiment must make the required correction explicit instead of treating a formally valid model as an operationally valid simulation.

The second goal is to measure how well the resulting simulation reproduces an unseen period of terminal operation. This includes the activity sequence, service times, truck turnaround times, and the arrival process. Three ProSiT configurations are compared to test whether additional routing and workload context improves held-out fidelity. This is tested directly. More context is treated as a modelling choice, not as an expected improvement.

The third goal is to examine the model as a white-box tool for operational what-if analysis. The selected model is exposed to a resource-pool change at T22 and to a higher truck-arrival rate. These interventions test whether the specified input changes reach the simulation and whether the resulting KPI changes are interpretable within the model boundaries. They do not establish that the same effects would occur in the real terminal.

Across these goals, robustness is assessed through a temporal hold-out, repeated simulation runs, model-quality diagnostics, and sensitivity checks. Together, these experiments provide the evidence required to answer the research questions introduced in Chapter 1.

4.2 SOLUTION ARCHITECTURE

The solution architecture is organised as a sequence of persistent processing blocks. Each block produces an artefact that is used by the next block. This makes the split, model discovery, simulation, and evaluation reproducible and prevents information from the test period from entering model discovery. Figure 4.1 gives an overview of the complete pipeline.

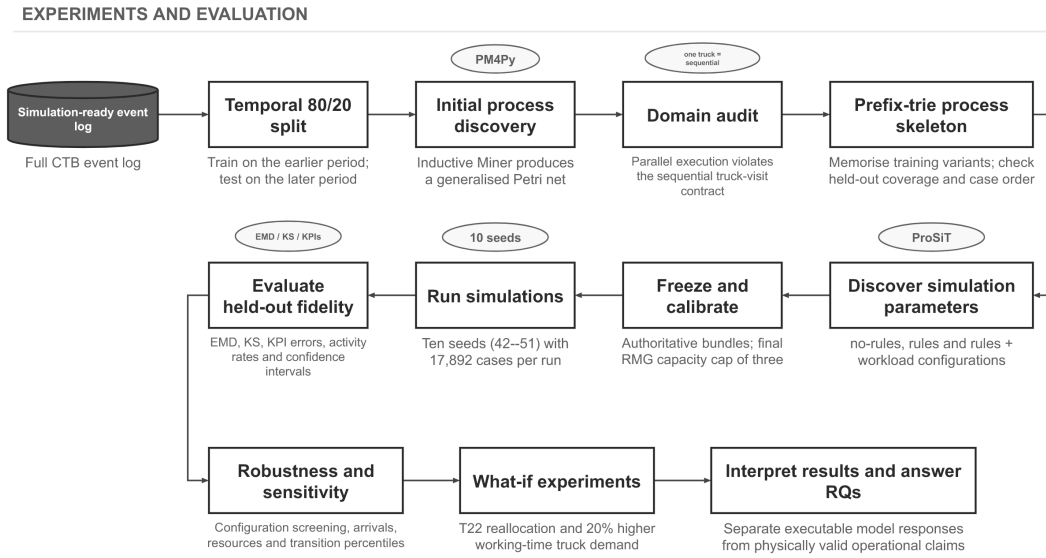


Figure 4.1: Solution architecture of the experimental pipeline.

The input is the simulation-ready event log from Chapter 3. It is divided chronologically into a training partition and an unseen test partition. Only the training data are used to discover the control flow and the ProSiT simulation parameters. The initial Inductive-Miner model is inspected against the physical truck-visit contract. Because this model permits parallel execution within one case, it is replaced by a prefix-trie Petri net that preserves the activity order of the observed training variants. This net becomes the common control-flow skeleton for the simulation configurations.

ProSiT then discovers arrival, timing, routing, and resource parameters from the training partition. The three configurations differ in the amount of context made available to their decision models. Each discovered parameter bundle is stored separately and simulated over the same time window and case volume as the test partition. Repeated runs with different seeds pro-

duce the simulation logs used for the fidelity comparison and confidence intervals.

The evaluation block compares the simulated logs with the held-out event log from several perspectives. It checks control-flow coverage, timing distributions, point statistics, arrival behaviour, and resource assumptions. After this historical evaluation, one transparent parameter bundle is used for the two what-if experiments. The scenario runner changes only the defined input parameters and compares each intervention with a matched-seed baseline. The transition-duration sensitivity study and the documented ProSiT workarounds provide additional diagnostic evidence around the main pipeline.

4.3 HELD-OUT TRAIN/TEST SPLIT

4.3.1 RATIONALE

A held-out validation set is needed to answer RQ1–RQ3. In simulation discovery it is not sufficient to report the fit of parameters on the data used to discover them: a well-fit ProSiT bundle can memorise the training distribution while over-fitting the operational dynamics. The ProSiT reference implementation [?] describes an in-memory split in its *Advanced Usage* example, but does not persist the split, does not commit to a case-level boundary and does not report validation metrics. This thesis uses a persistent, case-level, temporally ordered split that any downstream script can consume unchanged.

4.3.2 SPLIT DEFINITION

Let $\mathcal{L} = (c_1, c_2, \dots, c_N)$ be the set of $N = 89,460$ cases in the final event log. For each case c_i let $\tau(c_i)$ denote its earliest event timestamp:

$$\tau(c_i) = \min\{\text{start}(e), \text{time}(e) \mid e \in c_i\}. \quad (4.1)$$

Sort the cases by τ and a deterministic case-identifier. The first 80 % form the training set and the remaining 20 % form the test set. The cutoff τ^* is recorded as the earliest arrival in the test set.

On the CTB event log the cutoff is $\tau^* = 2026-04-20\ 18:17:00$. The split produces 71,568 training cases (220,048 events) and 17,892 testing cases (55,360 events); no events are dropped. The

split is realised by `validation/01_train_test_split.py` and is documented in `validation/results/sp` which records the seed, the cutoff and the per-activity coverage in both partitions. The manifest is loaded by every downstream simulation script so that n_{traces} and t_{start} match the test window exactly.

4.4 PROCESS DISCOVERY

4.4.1 EXPECTED PROCESS-DISCOVERY

In a data-aware process-simulation workflow, the control flow is normally discovered from the event log before the timing, routing, arrival, and resource parameters are fitted. The expected output is a process model that reproduces the recorded behaviour and also abstracts beyond individual traces. A useful discovered model balances fitness, precision, generalisation, and simplicity [? ?].

For ProSiT, this process model also has a practical role. Its Petri net becomes the executable backbone of the simulation. Formal soundness and good conformance are not enough for this role. Every execution allowed by the model must also make sense for one physical truck visit. The CTB contract is simple: one `Gate In`, one or more sequential yard activities, and one final `Gate Out`. Activities of different trucks may run in parallel. The activities of one truck may not.

The work started with the standard discovery step. The resulting model was then checked against this domain contract before it was passed to ProSiT.

4.4.2 INITIAL INDUCTIVE-MINER DISCOVERY

An Inductive-Miner model was generated from the training log. This was the expected control-flow input for ProSiT. The Inductive Miner was selected because it produces sound, block-structured models and turns individual traces into a more general behavioural model [?]. The initial result was inspected as a process tree and as the equivalent BPMN model in Figure 2.2.

This was also a check of the event-log representation. The initial PM4Py conversion ordered events by their completion timestamps. Some CTB timestamps overlap slightly. This can move `Gate Out` ahead of a yard event in the ordered trace, even though the truck operation has a

fixed logical order. The miner then treats this ordering as process evidence. A sound process model can still be unsuitable for simulation when the event order does not match the physical control order.

4.4.3 DOMAIN CHECK AND PARALLEL-EXECUTION PROBLEM

The first domain check found such a conflict. The process tree starts with `Gate In` and then contains an `and cut`. This cut places `Gate Out` in parallel with the yard subtree, as shown in Figure 2.2a. Under the semantics of an `and cut`, the two branches may interleave. The model language admits an order such as `Gate In`→`Gate Out`→yard activity. Inner `and cuts` can also enable more than one yard branch for the same case.

These executions violate the CTB physical constraints. A truck cannot leave before its yard work is complete. It also cannot perform two yard activities at the same time. This finding comes from the operator structure of the discovered tree. It is not a conclusion from a low conformance score.

The distinction matters here. The Inductive-Miner model is formally valid for the event order presented to it. Its language is too permissive for the intended simulation. Process-discovery quality and domain-valid simulation behaviour are related, but they are not the same requirement. The initial Inductive-Miner model was kept as evidence of this problem. It was not used as the final ProSiT backbone.

4.4.4 ORDER CORRECTION AND MANUAL PREFIX-TRIE IMPLEMENTATION

The pipeline was changed in two places. First, `_eventlog_contract.py` creates a logical case order that keeps `Gate In` first, the recorded yard-event order in the middle, and `Gate Out` last. It converts this ordered table directly to a PM4Py event log. It does not call the formatter that sorts again by completion time. The original start and completion timestamps remain unchanged for timing and resource discovery.

Second, the executable control flow was replaced with a raw sequential prefix trie. This was a manual modelling decision. The trie itself is built automatically from the 70 variants in the ordered training log; the variants were not entered by hand. The builder is also implemented in `_eventlog_contract.py` and is called by `baseline/07_run_prosit_discovery.py`.

Each observed prefix becomes a place in the Petri net. Each observed next activity becomes a

labelled transition to the next prefix. Shared prefixes are stored once. Every complete path ends in one common sink after `Gate Out [? ? ]`. The resulting net is acyclic and uses one case token. It contains 76 places and 144 transitions.

The implementation combines the observed training language with the explicit truck contract. It accepts only paths with one initial `Gate In`, at least one sequential yard activity, and one final `Gate Out`. This restriction applies inside one case. It does not prevent different trucks from using resources at the same time. Cross-case concurrency is handled by the resource models and the capacity constraints.

Correcting the event order removes the timestamp-sorting problem. The trie adds a stricter guarantee: the executable model can only follow a full sequence seen in training. This solves the sequencing problem by giving up structural generalisation. The trie cannot create an unseen activity combination, loop, or exception path.

The final control flow is not an automatically discovered general process model. It is a data-derived and domain-constrained execution contract. ProSiT still discovers the arrival, duration, routing, resource-selection, and other simulation parameters. The final simulator is only partially automatically discovered. Its valid case order comes from observed variants plus domain knowledge, not from automatic discovery alone.

4.4.5 HELD-OUT COVERAGE AND CASE-CONTRACT VALIDATION

The trie is evaluated as a simulation contract, not as a successful general process-discovery model. The standard quality measures are still reported for the training and test partitions:

- token-based replay fitness;
- token-based replay precision as implemented by `PM4Py`;
- generalisation via `pm4py.generalization_tbr`; and
- simplicity via `pm4py.simplicity_petri_net`.

Their interpretation differs from a normal discovered model. Training fitness is trivial because each training variant is encoded. Precision, generalisation, and simplicity are reported as model descriptors. They are not used to claim that the trie is a high-quality discovery result.

The main test is held-out sequence coverage. A fitting held-out case is a later truck visit whose complete activity sequence already occurs in the training trie. This test asks whether the restrictive language remains usable in the later period. High coverage would show repeated CTB paths in this dataset and time window. It would not show that the trie generalises.

The generated logs are also checked against the case contract. Each case must contain one initial Gate In, at least one yard activity, and one final Gate Out. Activities inside one case may not overlap, and completion timestamps may not decrease. This separates replay coverage from physical sequence validity. The protocol is implemented in `baseline/07_run_prosit_discovery.py` and `_eventlog_contract.py`. Its results are reported in Section 5.2.

4.5 SIMULATION PARAMETER DISCOVERY CONFIGURATIONS

ProSiT is used unchanged as the discovery and simulation engine [?]. All simulation parameters are learned by `SimulatorParameters.discover_from_eventlog` from the training partition only. Three configurations define an increasing contextual- expressiveness ladder. The retained screening results compare the two endpoints, *no-rules* and *rules+workload*. This measures the combined effect of the additional modelling machinery. Because the intermediate *rules* result is not retained in that comparison, the separate contribution of workload features is not inferred.

4.5.1 CONFIGURATION *NO-RULES*

The *no-rules* configuration uses `max_depth_tree= 0`, which disables the ProSiT decision-tree mechanism entirely. Every parameter reduces to a single unconditional distribution or a marginal frequency: one arrival-rate distribution, one execution-time distribution per activity, one waiting-time distribution per resource, one categorical distribution over routing choices at each Petri-net decision point. This configuration is the distribution-based reference point for RQ₃.

4.5.2 CONFIGURATION *RULES*

The *rules* configuration uses the ProSiT default (`max_depth_tree= 3`, `min_samples_leaf_cv= [50, 100, 200]`, `random_state= 42`). Every timing, routing and resource-selection model is

fitted as a decision tree whose leaves carry the local distributions. Cross-validation over the leaf-size grid selects the actual leaf size per model, and models that fail to beat the marginal distribution on held-out CRPS collapse to the pooled baseline (this is a ProSiT internal safeguard against overfitting to noise [?]). Case-level attributes made available to the trees are the visit-related, demand, utilisation and target-area features listed in Section 3.2.7.

4.5.3 CONFIGURATION *RULES+WORKLOAD*

The *rules+workload* configuration is identical to *rules* but with `use_workload_features=True`. As documented in the ProSiT README this enables two extra features on the waiting-time and resource-selection models: current workload (number of concurrent tasks) and queue length (number of tasks scheduled but not yet started) at the time each event is enabled in the process model. The intention behind including this configuration is to let ProSiT see the queueing state that its distribution family alone would otherwise ignore. It forms the more expressive endpoint of the retained RQ₃ comparison.

4.5.4 DISCOVERY HYPERPARAMETERS

Table 4.1 lists the ProSiT settings used in the three configurations. It also includes multitasking because this setting changes the resource semantics, even though it is fixed across the comparison. `attribute_mode` is set to 'empirical' for the reasons given in Section 4.10. All other arguments use the ProSiT defaults.

Hyperparameter	no-rules	rules	rules+workload
<code>max_depth_tree</code>	0	3	3
<code>min_samples_leaf_cv</code>	–	[50, 100, 200]	[50, 100, 200]
<code>use_workload_features</code>	False	False	True
<code>enable_multitasking</code>	True	True	True
<code>multitasking_thr</code>	0.05	0.05	0.05
<code>attribute_mode</code>	empirical	empirical	empirical
<code>random_state</code>	42	42	42

Table 4.1: ProSiT discovery hyperparameters by configuration.

With `enable_multitasking=True`, ProSiT can classify a resource as multitasking when at least 5 % of its observed events show concurrent work. It can then learn resource capacity from

overlaps between different truck cases. This is separate from the control-flow problem above. The prefix trie keeps the activities of one truck sequential. Multitasking allows a resource to serve several trucks at the same time. It is also different from `use_workload_features`, which only adds workload and queue state as inputs to the decision trees.

The full pipeline is executed by `baseline/07_run_prosit_discovery.py`. The pickled `SimulatorParameters` objects (`prosit_params.pkl`) and their JSON representations are stored in the paramset folder `baseline/discovery_params/params_<utc>_train80/prosit_discovery_no_rules|_workload|/`.

4.5.5 AUTHORITATIVE PARAMETER BUNDLE AND SERIALIZATION CONTRACT

In this thesis, *authoritative parameter bundle* is an operational reproducibility term: it denotes the exact executable `SimulatorParameters` state loaded by the validation and scenario scripts. If a derived table, JSON summary and executable state disagree, the serialized object that generated the reported simulations is the source of truth. Two provenance layers are distinguished. The frozen, training-only calibrated *rules+workload* `prosit_params.pkl` is authoritative for the freely discovered and calibrated source model. The final CTB baseline is a deep-copied effective bundle in which the RMG concurrency values are capped at the domain maximum of three; it is stored as `params_baseline_rmg_max_concurrency_3.pkl` in the final scenario result directory and is authoritative for the final validation and scenario results. The discovered overlap capacities remain auditable without being mistaken for physically feasible capacities. Both objects contain the Petri net and markings, routing and resource-selection rules, resource mappings and capacities, calendars, distribution objects, calibration metadata and the empirical sample arrays used for stochastic draws.

The ProSiT JSON export is retained as a portable, human-inspectable companion, but it is not an execution-equivalent serialization of the calibrated object. The JSON rule nodes preserve fitted distribution names, parameters and representative values but do not serialize all empirical sampled arrays held in the Python object. This matters because the CTB calibration modifies those arrays and ProSiT samples from them at runtime. Reconstructing the final simulator from JSON alone can change its stochastic behaviour. The pickle is authoritative because the JSON does not fully represent all empirical sample arrays; this statement concerns the actual ProSiT serialization schema used here, not an intrinsic superiority of the pickle format.

The companion files `prosit_run_summary.json`, `control_flow_contract.json` and `ctb_calibration.json` document provenance and intended changes. Validation and scenario scripts record SHA-256 hashes of their source pickles, and scenario bundles are produced by deep copy rather than by mutating the frozen baseline. Pickle is Python- and class-version-specific and must never be loaded from an untrusted source. Reproducibility requires the recorded software environment. These limitations are the reason for retaining the JSON companions even though they are not the authoritative runtime state.

4.6 MONTE-CARLO REPLICATION PROTOCOL

Every ProSiT simulation includes stochastic draws from the fitted distributions. One Monte-Carlo draw is not enough to conclude that one configuration is more faithful than another: the observed difference may be within the sampling variance of the simulator itself. This is the same many-possible-realizations problem addressed by the multi-log validation design of Chapela-Campa et al. and by classical independent-replication output analysis [? ?]. This thesis reports every fidelity metric with a Monte-Carlo confidence interval.

4.6.1 REPLICATION DESIGN

The same replication design is used for the three ProSiT configurations $c \in \mathcal{C} = \{no\text{-}rules, rules, rules+workload\}$. For each configuration, the stored parameter bundle is loaded and a fresh simulation engine is created. Ten runs are then generated with the seeds $s_i = 42 + i$ for $i = 0, \dots, 9$. Each run starts at the test cutoff $t_{start} = 2026-04-20\ 18:17:00$ and generates $n_{traces} = 17,892$ cases. The start time and case count are read from the split manifest so that every simulated log represents the same window as the held-out data.

Let $m_{c,i}$ be one fidelity metric for configuration c and seed s_i . The reported value is the mean across the $N = 10$ runs. Its 95 % Student-t confidence interval is calculated as

$$\bar{m}_c = \frac{1}{N} \sum_{i=0}^{N-1} m_{c,i}, \quad CI_{95\%} = \bar{m}_c \pm t_{0.975, N-1} \frac{s_{m,c}}{\sqrt{N}}, \quad (4.2)$$

where $s_{m,c}$ is the sample standard deviation across the runs. This keeps the comparison identical across configurations and separates the reported mean from the simulator's run-to-run

variation.

4.6.2 IMPLEMENTATION

The protocol is implemented in `validation/06_multi_seed_ci.py`. $N_{\text{seeds}} = 10$ is used for every configuration. This mirrors the $K = 10$ design used by Chapela-Campa et al. while remaining computationally feasible for the 17,892-case CTB test window [?]. The resulting Student-t intervals quantify simulation sampling uncertainty conditional on one frozen model and one fixed test log; with only ten replications, they are not estimates of discovery or input-model uncertainty. The base seed is 42.

The choice to seed Python’s `random` and `numpy.random` before each `engine.apply()` rather than the ProSiT engine itself is intentional: ProSiT 1.0.3 does not expose a per-call seed on its simulation engine. Seeding both random-number generators before the call yields reproducible replications and captures the same stochastic variability that a user would observe in production.

4.7 EVALUATION METRICS

The comparison of a simulated event log against the real held-out log uses four distributional distance metrics. Each metric is computed once per Monte-Carlo replication so that a confidence interval can be attached to the point estimate.

4.7.1 WASSERSTEIN DISTANCE (EMD)

For each yard activity a the service-time EMD is

$$\text{EMD}_a^{\text{svc}} = W_1(F_a^{\text{sim,svc}}, F_a^{\text{real,svc}}), \quad (4.3)$$

where $F_a^{\text{sim,svc}}$ and $F_a^{\text{real,svc}}$ are the empirical CDFs of time—start for activity a in the simulated and real log, respectively. Gate In and Gate Out are excluded from the service-fidelity aggregation because both have zero-duration timestamp constructions and would mechanically improve the score without representing terminal handling. Let $\mathcal{A}_Y$ be the set of yard activities and n_a^{real} the number of valid held-out observations. This thesis reports all per-activity values and two

yard-level aggregates:

$$\overline{\text{EMD}}_Y = \frac{1}{|\mathcal{A}_Y|} \sum_{a \in \mathcal{A}_Y} \text{EMD}_a^{\text{svc}}, \quad (4.4)$$

$$\overline{\text{EMD}}_{Y,w} = \frac{\sum_{a \in \mathcal{A}_Y} n_a^{\text{real}} \text{EMD}_a^{\text{svc}}}{\sum_{a \in \mathcal{A}_Y} n_a^{\text{real}}}. \quad (4.5)$$

The first gives every yard-activity class equal influence; the second describes the error experienced by the empirical mix of yard events. The case-level turnaround EMD is $W_I(F^{\text{sim},\text{ttot}}, F^{\text{real},\text{ttot}})$ where ttot is the total elapsed time per case (first start to last complete). The inter-arrival EMD is $W_I(F^{\text{sim},\text{iat}}, F^{\text{real},\text{iat}})$ where iat is the time between consecutive case-arrival timestamps. All EMDs are reported in minutes. The main view retains durations up to 24 hours and is accompanied by an uncapped raw view so that an artificial simulation tail cannot be hidden.

4.7.2 KOLMOGOROV–SMIRNOV STATISTIC

For each distribution pair the two-sample KS statistic is computed by `scipy.stats.ks_2samp`. Because the CTB test log contains 55,360 events, sub-sampling is applied so that each side contributes at most 10,000 observations to the test. This limits computational cost and the extreme sample-size sensitivity of the associated significance test; it does not turn the p-value into a fidelity effect size. The KS statistic (magnitude of the maximum vertical CDF gap) is reported alongside the EMD as the main measure; the p-value is retained only as a secondary diagnostic.

4.7.3 POINT-STATISTIC ERRORS

For each activity and each case-level KPI the mean, median and 90th percentile of the simulated and real distributions are compared as absolute and relative errors. This complements the distributional metrics with operationally interpretable point statistics (e.g. “the simulator under-predicts mean turnaround by X minutes”).

4.7.4 IMPLEMENTATION

The metric definitions are implemented in `validation/02_validate_simulation.py` and reused verbatim by `validation/06_multi_seed_ci.py` to guarantee that single-shot and multi-seed numbers are directly comparable.

4.8 WHAT-IF SCENARIO DESIGN

Two independent interventions are evaluated to address RQ4. Both are derived by deep copy from the frozen, training-only calibrated *rules+workload* baseline. No held-out observation is used to select or fit a scenario parameter.

4.8.1 SCENARIO A: T22 RESOURCE-POOL REALLOCATION

Scenario A removes T22 from the simulated RMG resource pool. The scenario is constructed by (i) removing T22 from every `params.act_to_resources` entry that references it, after verifying that each affected activity retains at least one eligible resource; and (ii) setting all 168 weekly calendar slots of T22 to unavailable. All other parameters are unchanged.

This is a resource-pool reallocation experiment, not a complete spatial yard-closure model. The discovered selector may use target-area attributes as statistical predictors, but ProSiT does not impose a hard constraint that binds a container to its historical storage block. A truck whose historical target was T22 can be assigned to another eligible block without an explicit container-relocation operation or a location-dependent travel-time adjustment. This boundary is discussed in Section 5.9.

4.8.2 SCENARIO B: 20 % HIGHER TRUCK DEMAND

Scenario B changes only the empirical working-time inter-arrival distribution. For demand multiplier $m = 1.20$, every empirical inter-arrival sample x_i is replaced by x_i/m . This preserves the zero mass and the empirical distribution shape while increasing the modelled arrival intensity by exactly 20 %. The gate-admission calendar, process model, duration models, routing, resource mappings, resource calendars and case-attribute schema remain unchanged. The scenario is independent of the T22 resource-pool reallocation.

4.8.3 FINAL DOMAIN-CONSTRAINED RMG CAPACITY

The freely discovered serialized source assigns maximum concurrency four to ten RMG block identifiers and five to twelve, for an aggregate effective upper bound of 100. ProSiT obtains these numbers from overlapping timestamps; they are empirical concurrency proxies rather

than equipment counts. They exceed the hard physical upper bound of three cranes and the three lanes in which trucks can wait after announcing readiness at a block. The final effective baseline replaces every RMG value c_r with $\min(c_r, 3)$. The aggregate upper bound becomes $22 \times 3 = 66$. This does not assert that three cranes normally serve landside trucks: in regular operation two cranes may serve the waterside and at most one may serve the landside, but that allocation depends on ship operations and is not observed in this event log. Three is a reasonable physical ceiling, not an estimated landside staffing level. No resource mapping, calendar, selection rule, timing model, arrival model or control-flow parameter is changed. The T22 and demand interventions are then independently re-derived from this capped baseline.

This is a transparent domain constraint rather than a data-driven recalibration: the freely discovered source remains frozen as discovery evidence, whereas the cap-three copy is the authoritative effective CTB baseline. An uncapped run is retained only as a diagnostic sensitivity to show what the overlap-derived parameter would imply. Both sets of replications use identical seeds, trace counts and start time. In addition to direct cap effects, the diagnostic comparison reports the interaction

$$\Delta_{\text{interaction}} = (Y_{\text{scenario}} - Y_{\text{baseline}})_{c \leq 3} - (Y_{\text{scenario}} - Y_{\text{baseline}})_{\text{discovered}}, \quad (4.6)$$

so that changes in scenario response are not confused with a general level shift caused by the cap.

4.8.4 REPORTING

The unified driver `validation/09_multi_seed_scenarios.py` executes baseline and both interventions with ten matched seeds (42–51), $n_{\text{traces}} = 17,892$ and $t_{\text{start}} = 2026-04-20\ 18:17$. All paired deltas use common random numbers. Mean, median and 90th percentile of turnaround, RMG service and RMG pre-service delay are reported together with the observed arrival KPIs. For the T22 experiment, service and pre-service effects are reported separately for `RMG_receive` and `RMG_delivery`. Every one of the 30 generated logs is subjected to the sequential case contract of Section 4.4; any violation aborts the run. A separate diagnostic invocation with `--no-rmg-cap` reproduces the 30 uncapped overlap-capacity replications. `validation/10_compare_rmg_cap` verifies identical source hashes and settings before computing the paired direct effects and interactions. These uncapped results are diagnostic only and are excluded from the authoritative final result table; the 30 cap-three scenario logs form the final scenario evidence.

4.9 DESCRIPTIVE TRANSITION-DURATION ANALYSIS

As discussed in Section 3.2.5, an earlier pipeline version reconstructed activity enablement from route-specific transition baselines. Although this approach was abandoned in favour of ProSiT's internal model-derived enablement, the empirical transition durations remain informative as descriptive evidence of the physical separation between terminal activities.

To characterise this physical component, the same baseline table is recomputed with the 5th, 10th, 25th and 50th percentiles as alternative choices. For each transition (a, b) with at least twenty observations the resulting baseline duration is reported together with the shift relative to the 5th percentile and the rank correlation between percentiles. This analysis is implemented in `validation/04_baseline_percentile_sensitivity.py` and reported in Section 5.7. It does not affect ProSiT's simulation-parameter discovery; its purpose is to illustrate that the model-derived pre-service delay includes a non-trivial physical-movement component that cannot be separated from operational waiting with the available data.

4.10 PROSiT IMPLEMENTATION NOTES AND WORKAROUNDS

Applying ProSiT 1.0.3 [?] to the CTB event log uncovered several implementation issues that would have propagated as silent misbehaviour if not addressed. This section documents them so that the experiments can be reproduced deterministically and so that the downstream fidelity numbers can be interpreted correctly.

4.10.1 DATA-ATTRIBUTE DISCOVERY ON NAN COLUMNS

The default `attribute_mode='distribution'` passes each case attribute through a per-attribute distribution fitter. In the CTB log, `full_ratio` and `is_full` contain NaN for cases that involve empty-container-only visits. The distribution fitter in ProSiT 1.0.3 forwards the raw values to `scipy.stats`, which raises `TypeError: ufunc 'isfinite' not supported on the mixed dtype`. Switching to `attribute_mode='empirical'` bypasses the per-attribute fitter and samples full attribute tuples from the observed joint distribution instead. This is not a fidelity compromise: the discovered simulator still preserves case-level attribute correlations, and empirical sampling from the training distribution is the appropriate choice given the num-

ber of categorical attributes present in the CTB log.

4.10.2 SIMULATION IN NO-RULES MODE

In *no-rules* mode ProSiT samples the routing choice via `random.choice(seq)` where `seq` is a `numpy.ndarray`. Python's built-in `random.choice` evaluates `if not seq:`, which raises `ValueError: The truth value of an array with more than one element is ambiguous.` on multi-element numpy arrays. The workaround is a minimal monkey patch that intercepts `random.choice` while the engine is running and forwards `numpy.ndarray` arguments through `numpy.random.randint`. The patch is contained to the simulation call and is restored immediately afterwards. It is applied by both `baseline/07_run_prosit_discovery.py` and `validation/06_multi_seed_ci.py`.

4.10.3 DUPLICATED COLUMNS IN THE SIMULATED LOG

ProSiT 1.0.3 writes case-level attributes into the event-level frame returned by `SimulatorEngine.apply()`. When a case-level attribute has the same name as an event attribute (in the CTB log this happens for `enabled:timestamp` because ProSiT writes case-level attributes into the event-level frame), the resulting `pandas.DataFrame` carries duplicate column labels. Standard operations such as `pd.to_datetime(df[col])` then return a `DataFrame` slice instead of a `Series` and fail. The workaround is a single `df.loc[:, ~df.columns.duplicated()]` call at the start of every KPI-computation function. It is applied both in the what-if scripts and in the multi-seed CI driver.

4.10.4 TEMPORAL FEATURES OUTSIDE THE PROSiT INPUT

The final discovery contract does not pass hour-of-day or day-of-week attributes to ProSiT. It keeps the five standard event-log columns and the approved visit-related, demand, utilisation and target attributes from Table 3.8. Other columns are removed before simulation-parameter discovery. Hour and weekday are derived from timestamps only in the separate arrival and explanatory analyses. The retained ProSiT configurations therefore do not condition their timing rules directly on calendar features. This is an input-model boundary when interpreting the day/night differences reported in Section 5.5.

4.10.5 NON-DETERMINISTIC SIMULATION ENGINE

`SimulatorEngine.apply()` does not accept a seed argument. The Monte-Carlo protocol of Section 4.6 seeds Python's `random` and `numpy.random` before every call to `apply()`. Two consecutive calls with different seeds produce different logs; two consecutive calls with the same seed produce the same log up to floating-point noise. This is the reproducibility guarantee assumed throughout Chapter 5.

4.10.6 CONTRIBUTION BOUNDARY

The ProSiT engine itself was not modified for these workarounds. The CTB pipeline patches its own callers so that ProSiT can be invoked without source-code changes. The workarounds and the documented input boundary constitute the technical adaptation of ProSiT to the CTB case study. They are listed here so that a reader can distinguish the contribution of this thesis (case-study specific data engineering, held-out validation methodology, multi-seed replication, sensitivity analysis, what-if scenario protocol) from the contribution of ProSiT itself (automatic simulation-parameter discovery from event logs [?]).

5

Results

This chapter reports the results for the four research questions from Chapter 4. RQ₂ comes first. The control-flow contract must be usable before timing can be evaluated. RQ₁ then compares the final model with the temporal hold-out. RQ₃ compares the two retained ProSiT configurations. The next sections examine context, arrivals, resources, and transition times. These diagnostics help interpret the results, but they do not prove their causes. RQ₄ evaluates the two model interventions.

The chapter uses three types of evidence: real-versus-simulated hold-out comparisons, comparisons between configurations, and paired scenario changes. They answer different questions. The limitations are discussed before the final answers to RQ₁–RQ₄.

5.1 TEMPORAL TRAIN-TEST SPLIT

The temporal split contains 89,460 cases. The first 71,568 cases form the training log. They contain 220,048 events and cover Gate In arrivals from 2026-03-02 05:27 to 2026-04-20 18:15. The next arrival at 2026-04-20 18:17 starts the test log. It contains 17,892 cases and 55,360 events. No event is removed because of a missing timestamp. Table 5.1 gives the activity counts.

All activities also occur in the test log, but their frequencies change. LL_delivery rises from

Activity	Train	Test	Total
Gate In	71,568	17,892	89,460
Gate Out	71,568	17,892	89,460
RMG_receive	32,794	7,171	39,965
RMG_delivery	17,823	4,681	22,504
RMG_mixed	3,986	953	4,939
LL_delivery	9,612	3,618	13,230
HO ₂ _delivery	5,775	1,449	7,224
HO ₂ _receive	4,188	1,276	5,464
HO ₂ _mixed	749	276	1,025
LL_receive	1,932	141	2,073
LL_mixed	53	11	64

Table 5.1: Activity counts on the train and test partitions.

0.134 to 0.202 events per case. RMG_receive falls from 0.458 to 0.401, and LL_receive falls from 0.027 to 0.008. This shift is kept in the evaluation. It contributes to the final yard-activity-rate L1 error in Table 5.3. A random split would mix this later-period shift into both partitions. The temporal split does not. It still covers only one later CTB period.

5.2 CONTROL-FLOW STABILITY ACROSS THE HELD-OUT PERIOD

The final simulator uses the raw prefix trie from Section 4.4. It has 76 places and 144 transitions for 70 training variants. It is not a general process abstraction. It stores the observed sequence language. Training fitness and precision of one follow from this construction. They are not results. Table 5.2 also reports the held-out coverage and structural measures.

The held-out result is more useful. The trie accepts only training variants. The test log contains 53 variants, of which 43 were seen in training. These 43 variants cover 17,882 of 17,892 test cases. The other ten cases each follow a different unseen variant. Only 0.0559 % of test cases lie outside the stored training language.

The high fitness is not the main result here. The trie is intentionally designed to fit known variants. The main result is the repetition in the CTB data. A model that cannot create a new route still covers almost the full test period. Activity frequencies change between train and test,

Metric	Train	Test
Log fitness	1.0000	0.9998
Fitting traces (%)	100.000	99.944
Average trace fitness	1.0000	0.9990
Token-based precision	1.0000	0.9605
Generalisation	0.5767	0.3732
Simplicity	0.6179	0.6179

Table 5.2: Prefix-trie conformance and structural descriptors on train and test.

as shown in Section 5.1. The complete activity orders hardly change.

This restriction has a clear cost. Test generalisation is 0.37327 and simplicity is 0.61798. The trie is not compact and cannot generate unseen combinations. It does provide a strict case-order contract: one Gate In, one or more sequential yard activities, and one final Gate Out. Different trucks can still use resources at the same time. The path of each case must have occurred in training.

The model can test timing, resources, and arrivals on these historical paths. It cannot support a claim that a general CTB process model was discovered. It also cannot create a new exception path, handling step, or activity order. A structural response to a disruption is missing unless the same sequence already happened during training.

RQ₂ answer. The trie covers 99.944 % of test cases and enforces the required order. This shows that the recorded CTB trace language is highly repetitive. It does not show that the trie generalises. The trie is a useful execution contract for simulations on known paths. It does not cover new truck-process structures.

5.3 HISTORICAL FIDELITY AND MODEL SCREENING

This section contains two evaluations. RQ₁ compares the final calibrated *rules+workload* model with the temporal hold-out. RQ₃ compares the pre-calibration *no-rules* and *rules+workload* endpoints. These comparisons use different model states and must not be mixed.

5.3.1 RQ1: HISTORICAL FIDELITY OF THE FINAL BASELINE

The final baseline uses the prefix-trie contract, training routing marginals, the minute-resolved arrival model, and non-pre-emptive completion. The raw RMG capacities are capped at three per block. This cap comes from the domain, not from the hold-out. The hold-out is used only for evaluation.

Ten runs of 17,892 cases use seeds 42–51. Each run starts at the frozen cutoff $t_{\text{start}} = 2026-04-20\ 18:17$. Table 5.3 summarises the comparison. Its intervals cover seed variation for this fixed model and hold-out only.

KPI	Real/ref.	Simulated	Difference	95 % MC CI
Yard service EMD, unweighted (min)	–	2.999	–	[2.888, 3.110]
Yard service EMD, real-frequency weighted (min)	–	2.340	–	[2.279, 2.402]
Case-turnaround EMD (min)	–	9.306	–	[9.229, 9.384]
Case-turnaround KS	–	0.1554	–	[0.1517, 0.1591]
Inter-arrival EMD (min)	–	0.0724	–	[0.0674, 0.0774]
Inter-arrival KS	–	0.0254	–	[0.0215, 0.0292]
Mean turnaround (min)	39.742	30.436	–9.306	[30.359, 30.514] ^a
P90 turnaround (min)	71.000	51.800	–19.200	[51.498, 52.102] ^a
Simultaneous-arrival share (%)	56.660	54.221	–2.439 pp	[54.035, 54.407] ^a
Yard-activity-rate L1 error	–	0.1866	–	[0.1838, 0.1893]

Table 5.3: Held-out fidelity of the final calibrated cap-three baseline ($N_{\text{seeds}} = 10$, seeds 42–51; 17,892 cases per run).

^aThe interval refers to the simulated outcome. The real hold-out statistic is fixed. Gate In and Gate Out are excluded from both yard-service EMD aggregates.

The result is mixed. Inter-arrival EMD is 0.0724 minutes. Unweighted yard service EMD is 2.999 minutes. Frequency weighting reduces it to 2.340 minutes because some of the larger errors occur in rare activities (Table 5.4). The complete visit is less accurate. Mean turnaround is 9.306 minutes too short, or 23.42 %. P90 is 19.200 minutes too short, or 27.04 %. This bias remains across the ten seeds. The narrow seed intervals do not show that the model itself is correct.

All ten runs pass the case-order checks. They contain no gate-only cases, within-case overlaps,

invalid boundaries, or decreasing completion times. Gate Out never occurs before the last yard activity. No accepted duration exceeds 24 hours. These checks are necessary, but they do not remove the timing bias.

Yard activity	Mean EMD (min)	95 % MC CI
HO2_delivery	2.119	[2.013, 2.225]
HO2_mixed	4.255	[3.853, 4.656]
HO2_receive	2.024	[1.768, 2.280]
LL_delivery	1.353	[1.286, 1.420]
LL_mixed	3.857	[3.019, 4.695]
LL_receive	4.203	[4.000, 4.406]
RMG_delivery	2.519	[2.426, 2.612]
RMG_mixed	4.200	[3.971, 4.429]
RMG_receive	2.463	[2.374, 2.551]

Table 5.4: Held-out service-time EMD by yard activity for the final cap-three baseline ($N_{\text{seeds}} = 10$).

RQ1 answer. The model reproduces the case order and parts of the arrival and service-time behaviour. It does not reproduce the full turnaround distribution. Total visit time is systematically too short.

5.3.2 RQ3: CONTEXT AND HELD-OUT FIDELITY

Table 5.5 compares the two retained endpoints before the final CTB calibration. These are screening results, not the final RQ1 results.

Metric (min)	no-rules	rules+workload
Case turnaround EMD	13.94 [13.15, 14.73]	21.75 [20.60, 22.90]
Inter-arrival EMD	0.319 [0.317, 0.320]	0.314 [0.313, 0.315]
Case turnaround KS	0.103 [0.099, 0.108]	0.125 [0.123, 0.127]
Simulated mean turnaround	53.68 [52.89, 54.47]	61.49 [60.34, 62.64]
Simulated P90 turnaround	103.05 [101.26, 104.84]	124.44 [121.70, 127.17]

Table 5.5: Pre-calibration held-out screening by ProSiT configuration ($N_{\text{seeds}} = 10$; 95 % Student-t Monte-Carlo CI).

No-rules performs better on every turnaround measure. Its EMD is 36 % lower. In the other direction, *rules+workload* is 56 % higher. The seed intervals do not overlap. *Rules+workload* improves inter-arrival EMD by only 0.005 minutes, or 0.3 seconds. It is also less stable. Turnaround-

EMD standard deviation is 1.61 minutes instead of 1.10. P90 standard deviation is 3.82 minutes instead of 2.50.

These intervals describe seed variation for two fixed bundles. They do not include discovery uncertainty or another training period. The difference is stable in this screening run. It is not a population-level result.

The comparison has two limits. First, there is no *rules*-only result. The test covers the combined move from unconditional distributions to trees with workload features. It cannot isolate the workload effect. Second, both values are pre-calibration. They do not compare the final model with an equally calibrated *no-rules* model.

5.3.3 DELIBERATE CHOICE OF THE SCENARIO MODEL

No-rules is the better fidelity reference in this screening. The scenario experiment has another purpose. It tests whether the data-aware white-box model can be changed and inspected. The calibrated *rules+workload* model is used for that reason. It is not used because it predicted the hold-out better. RQ₄ tests white-box model experiments and their limits. It is not a test of the best CTB forecaster.

RQ₃ answer. More contextual expressiveness does not improve fidelity in the retained comparison. The tree-and-workload endpoint has worse turnaround results and more seed variation. The missing middle configuration prevents a separate conclusion about workload features.

5.3.4 MEANING OF THE TIME MEASURES

The real log has start and completion timestamps, but no enablement timestamp. Service-time EMD uses `complete - start`. Turnaround EMD uses `last_complete - first_start`. Both can be calculated for both logs. ProSiT derives enablement internally and writes it only to the simulated log. Simulated start — enabled is called pre-service delay. It cannot be validated against real waiting time.

5.4 CONTEXT SIGNAL IN ACTIVITY DURATIONS

The RQ₃ comparison shows that the more expressive endpoint is less accurate. The next diagnostic checks how much duration information is present in the available attributes. Table 5.6 reports a training-only decision-tree test. It is not a breakdown of the final EMD. It also cannot prove why the RQ₃ result occurs.

Activity	R ²
LL_delivery	0.263
LL_mixed	0.258
HO2_receive	0.115
HO2_mixed	0.070
RMG_mixed	0.051
HO2_delivery	0.029
LL_receive	0.024
RMG_receive	-0.005
RMG_delivery	-0.004
Gate In	1.000
Gate Out	1.000

Table 5.6: Per-activity duration R² of a decision-tree model conditioned on target demand, target utilisation, target rank, visit complexity, receive/delivery counts, hour, weekday and weekend indicator (training log; grid search over depth/leaf, best per activity).

Gate activities have $R^2 = 1.000$ because their service duration is fixed at zero. This is not useful predictive performance. The nine yard activities range from -0.005 for RMG_receive to 0.263 for LL_delivery. The two main RMG activities are close to zero. RMG activities still account for more than 60 % of yard events.

The available attributes explain little of the duration variation in the largest activity family. This is consistent with the weak RQ₃ result. It is not proof of its cause. Leaf size, fitted distributions, routing, resource selection, and the change between train and test may also affect the result.

5.5 ARRIVAL STRUCTURE AS A COUNTEREXAMPLE

The weak duration result does not mean that context is useless everywhere. Arrivals provide a counterexample. Table 5.7 compares one pooled arrival rate with a tree using hour, weekday,

and weekend features. The test uses a chronological 80/20 split inside the training partition. It is a training diagnostic, not another held-out validation result.

Model	MAE	RMSE	R ²
Pooled arrival rate	28.98	34.60	-0.001
Time-aware decision tree	9.41	16.87	0.762

Table 5.7: Arrival-count benchmark on 30-minute buckets (training log only, chronological 80/20 split).

The time-aware tree reduces MAE by 67.5 % and RMSE by 51.2 %. The main pattern is also visible in Table 5.8. The estimated day rate is about 8.7 times the night rate. Arrival intensity is not constant. Time conditioning is useful here. The result does not show that the tree captures every source of arrival variation.

Period	Mean inter-arrival (min)	Implied mean arrivals/hour
All	1.00	60.2
Day	0.54	110.9
Night	4.69	12.8

Table 5.8: Day and night arrival statistics on the training log.

The low inter-arrival EMD in Table 5.3 comes from the final calibrated simulator, not from this tree alone. More importantly, good arrival timing does not fix the turnaround bias. The missing time occurs after arrival.

5.6 RESOURCE DISCOVERY AND THE DOMAIN CAP

The discovered source bundle contains 26 resource identifiers. Table 5.9 shows their maximum concurrency. The source bundle is kept for audit. The final runs use a copy in which each RMG block is capped at three. All other parameters stay unchanged. The earlier pool-level report from `baseline/04_resource_concurrency_discovery.py` is only a diagnostic. It is not the simulator configuration.

All 168 weekly slots in each resource calendar are open. Gate closures are handled by the arrival calendar. Every RMG activity can select any of the 22 block identifiers. The learned rules provide the weights. Removing T22 leaves 21 identifiers and removes three capacity units from the final model.

Family	Identifiers	Eligible activities	Maximum concurrency
Gate In	Res.GateIn	Gate In	1
Gate Out	Res.GateOut	Gate Out	1
HO ₂	HO ₂	three HO ₂ activities	15
LL	LL	three LL activities	22
RMG	T ₀₆ –T ₂₇ (22 blocks)	three RMG activities	4–5 ^a

Table 5.9: Resource representation in the freely discovered calibrated source bundle before applying the final RMG cap.

^aTen RMG block identifiers have maximum concurrency four and twelve have maximum concurrency five; their aggregate upper bound is 100 simultaneous RMG tasks. This exceeds the nominal physical upper bound of 66 implied by at most three cranes at each of 22 blocks.

The raw RMG capacities sum to 100. The stated physical limit is 66: three cranes at each of 22 blocks. This mismatch is important. The discovered values describe overlapping event intervals. They are not verified crane counts. The cap removes the implausible upper bound. It does not prove that 66 is the effective capacity of the terminal.

This is the second main domain correction in the pipeline. The prefix trie was needed to enforce valid case order. The resource cap was needed to enforce a physical upper bound. The final simulator is a hybrid of discovered parameters and domain constraints. It is not an unchanged output of automatic discovery.

An RMG block is still only a selectable resource label. It is not container location state. After T₂₂ is removed, the model can assign work elsewhere without checking storage or travel feasibility. This limit is central to Scenario A and is discussed in Section 5.9.

5.7 SENSITIVITY OF TRANSITION-TIME BASELINES

An older pipeline version used route-specific transition baselines to reconstruct enablement. Section 3.2.5 explains why this was replaced by ProSiT’s own enablement. The old analysis is kept as a sensitivity check. It shows how much an assumed baseline changes with the chosen percentile. It does not separate movement, traffic, queueing, or other unrecorded delay.

The check uses percentiles {5, 10, 25, 50} and compares them with p₅. Table 5.10 covers the 48 transitions with at least twenty observations. The reported association is Pearson correlation.

Percentile	Mean (s)	Median (s)	Mean shift (s)	P90 shift (s)	Pearson r vs. p5
p5	225.8	223.5	0.0	0.0	1.000
p10	254.6	254.3	28.8	36.0	0.990
p25	318.3	311.0	92.5	110.3	0.948
p50	435.5	426.0	209.8	250.9	0.812

Table 5.10: Empirical transition duration under alternative percentiles (seconds; 48 transitions with ≥ 20 observations).

The values keep a similar ordering across percentiles. Pearson r is 0.990 between p5 and p10 and 0.812 between p5 and p50. Their absolute level still changes. From p5 to p50, the mean baseline rises by 209.8 seconds, or about 3.5 minutes. This is not a travel-time estimate. It shows that reconstructed waiting would depend on the selected percentile.

Figure 5.1 shows the CDF of the per-transition shift across the 48 retained transitions.

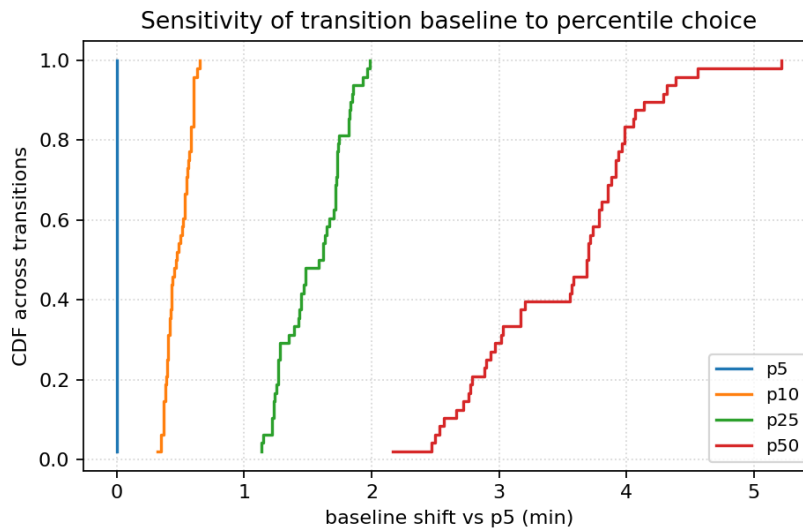


Figure 5.1: CDF of the per-transition duration shift relative to the 5th percentile, over the 48 transitions with at least twenty observations.

Interpretation. The data do not separate movement, traffic, queueing, and other delay. The real log also has no enablement timestamp. The model output is called pre-service delay. It is not validated queueing time.

5.8 WHAT-IF EXPERIMENTS

The scenario baseline uses the previously calibrated *rules+workload* bundle from RQ1 and the cap of maximum three available resources per RMG block. It is used because its rules can be inspected and changed. It was not the best screening configuration. Ten matched seeds (42–51) are run for the baseline, T22 removal, and the 20 % demand increase. This gives 30 simulations with 17,892 cases each.

Table 5.11 uses the baseline from this matched runner. It is only the reference for paired changes. The RQ1 results still come from Table 5.3.

The raw prefix trie also stays fixed. The experiments can change arrival timing, resource assignment, and model-derived delays. They cannot create a new activity sequence. Extra moves, detours, exceptions, or a new operation order are outside the model. All scenario results are conditional on the historical paths.

5.8.1 SCENARIO A: T22 RESOURCE-POOL REALLOCATION

T22 receives 604.8 assignments per baseline run. This is 4.41 % of RMG events. After removal, it receives none. The intervention executes as intended. The mean-turnaround change is +0.012 [−0.102, +0.127] minutes. Its paired interval includes zero. The intervals for mean RMG service and pre-service delay also include zero.

Receive service changes by −0.043 [−0.110, +0.024] minutes and receive pre-service by +0.016 [−0.039, +0.071] minutes. Delivery service changes by +0.035 [−0.078, +0.147] minutes and delivery pre-service by −0.036 [−0.087, +0.015] minutes. All four paired intervals include zero. The experiment does not show a clear difference between receive and delivery. This is not evidence that a physical T22 closure has no effect.

The experiment is not a full block-closure study. The remaining 21 blocks stay eligible for all RMG activities. Each keeps capacity three. The selector moves T22 work to another statistical resource. It does not check container location, block compatibility, relocation, or added travel. The result only tests the abstract resource pool.

Analysis	KPI	Reference	Outcome	Change	95 % CI
Scenario A: T22 resource-pool reallocation (scenario minus matched baseline)					
T22 pool	Mean turnaround (min)	30.443	30.456	+0.012	[−0.102, +0.127]
T22 pool	P90 turnaround (min)	51.700	51.900	+0.200	[−0.364, +0.764]
T22 pool	Mean RMG service (min)	11.069	11.059	−0.010	[−0.064, +0.045]
T22 pool	Mean RMG pre-service (min)	6.475	6.479	+0.003	[−0.034, +0.040]
T22 pool	RMG receive service (min)	11.362	11.319	−0.043	[−0.110, +0.024]
T22 pool	RMG receive pre-service (min)	6.480	6.496	+0.016	[−0.039, +0.071]
T22 pool	RMG delivery service (min)	8.794	8.829	+0.035	[−0.078, +0.147]
T22 pool	RMG delivery pre-service (min)	6.487	6.451	−0.036	[−0.087, +0.015]
T22 pool	Mean T22 assignments per run	604.8	0.0	−604.8	exact
Scenario B: 20 % higher working-time arrival intensity (scenario minus matched baseline)					
Demand	Model mean working inter-arrival (min)	0.6830	0.5692	−0.1138	exact
+20 %	Model arrivals per working hour	87.842	105.411	+20.000 %	exact
Demand	Observed mean inter-arrival (min)	0.8282	0.7145	−0.1137	[−0.1145, −0.1128]
+20 %	Observed arrivals per elapsed hour	72.452	83.979	+11.527 (+15.910 %)	[+11.478, +11.577]
Demand	Observed simultaneous-arrival share (%)	54.221	60.015	+5.795 pp	[+5.736, +5.853] pp
+20 %	Mean turnaround (min)	30.443	30.411	−0.033	[−0.128, +0.063]
Demand	Mean RMG service (min)	11.069	11.063	−0.006	[−0.067, +0.056]
+20 %	Mean RMG pre-service (min)	6.475	6.429	−0.046	[−0.094, +0.001]
Demand	RMG receive service / pre-service (min)	11.362 / 6.480	11.332 / 6.461	−0.030 / −0.020	[−0.101, +0.041] / [−0.074, +0.035]
+20 %	RMG delivery service / pre-service (min)	8.794 / 6.487	8.796 / 6.397	+0.002 / −0.090	[−0.083, +0.087] / [−0.157, −0.022]
Demand					
+20 %					
Structure	Aggregate RMG maximum concurrency	100 discovered	66 final	−34	exact domain cap
Contract	Invalid cases / duration tails	30 scenario logs	0	0	all checks passed

Table 5.11: Paired what-if scenario KPIs ($N_{\text{seeds}} = 10$, seeds 42–51; 17,892 cases per run).

Pre-service denotes the model-derived interval from enablement to activity start, not pure queueing time. “Observed” arrival measures use the minute-resolved simulated event log and elapsed calendar time. Intervals refer to paired seed-wise scenario-minus-baseline changes and quantify simulation-seed variation conditional on the frozen model.

5.8.2 SCENARIO B: 20 % HIGHER DEMAND

The working-time intervention is exact. Mean inter-arrival time changes from 0.6830 to 0.5692 minutes. The implied rate rises from 87.842 to 105.411 arrivals per working hour, or 20 %. The rate measured over elapsed calendar time rises by 15.910 %. Closed gate periods remain fixed. Timestamps are also floored to minutes. This flooring turns more short positive intervals into simultaneous observations. It does not change the zero mass in the input distribution.

Mean turnaround changes by -0.033 $[-0.128, +0.063]$ minutes. The interval includes zero. The same applies to mean RMG service, mean RMG pre-service delay, and the receive measures. Delivery pre-service falls by -0.090 $[-0.157, -0.022]$ minutes, or about 5.4 seconds. This small change is clear only for simulation-seed variation in this fixed model. The experiment does not identify its cause. It is not evidence of delivery prioritisation.

The missing congestion response is itself a model result. The cap of three does not create enough system interaction to raise turnaround under higher demand. The model has no explicit queue state, container location, or ship-side crane allocation. It cannot show that the real terminal could absorb 20 % more demand without congestion.

RQ4 answer. Both interventions execute as specified. T22 assignments fall to zero. Working-time arrival intensity rises by exactly 20 %. Both mean-turnaround intervals include zero. The model supports clear and repeatable experiments. It does not support a physical counterfactual or an operational recommendation from these two results. Executability is not counterfactual validity.

5.9 METHODOLOGICAL LIMITATIONS

5.9.1 PRE-SERVICE DELAY

ProSiT enables an activity when the process model allows it. Two consecutive CTB activities can occur at different locations. Truck movement in this Event log set up, is not a separate activity. The time from enablement (finishing a previous activity) to start can combine:

$$D_{\text{pre-service}} \approx T_{\text{physical}} + T_{\text{traffic}} + T_{\text{queue}} + T_{\text{other}}. \quad (5.1)$$

The log cannot identify these parts separately. The percentile check shows that a reconstructed baseline is sensitive to its assumption. It does not measure any one part. This would require position, trajectory, queue-entry, or similar state data. These data are not available. The results use the term *pre-service delay*, not queueing time.

5.9.2 RESOURCE ASSIGNMENT IS NOT SPATIAL CONTAINER RELOCATION

An RMG resource in ProSiT is an abstract capacity assignment. The discovered overlap capacity is 100. The stated physical cap is 66 across 22 blocks. This mismatch shows that timestamp overlap is not a crane count. The final baseline uses the cap of three per block.

The cap does not add spatial state. Target-area attributes are predictors, not hard inventory constraints. The simulator does not bind a container to a storage block. It also does not create relocation work after reassignment. Closing T22 spreads its statistical demand across the remaining blocks. It does not model feasibility, crane movement, or added truck travel. Scenario A tests the resource-pool abstraction. It does not forecast a physical T22 closure.

5.9.3 SCENARIO CONTROL FLOW IS RESTRICTED TO HISTORICAL VARIANTS

The prefix trie fixes control flow to the 70 training variants. This works for almost all cases in the later hold-out. It is more restrictive for scenarios. Every scenario case must follow a known path. A detour, repeated handling step, exception, or new activity order cannot occur. The two experiments only test parameter changes inside the historical trace language. They do not test a structural process response. Missing structural changes may also remove real sources of delay.

5.9.4 SYSTEMATIC TURNAROUND UNDER-PREDICTION

The final model under-predicts mean turnaround by 9.306 minutes and P90 by 19.200 minutes. The narrow seed intervals show that this is model bias, not simulation noise. The log cannot show how much missing time comes from movement, traffic, queueing, or an unrecorded state. Correct case order does not remove this limit on KPI forecasts.

5.9.5 SCENARIO CONFIDENCE INTERVALS EXCLUDE STRUCTURAL UNCERTAINTY

The matched-seed intervals in Table 5.11 cover random variation in one fixed model. They do not cover a new training period, another discovery run, or missing spatial state. The cap sets a ceiling but adds no location feasibility or equipment movement. More seeds would narrow these intervals. They would not reduce structural uncertainty. The experiments also use only the *rules+workload* configuration. They do not show whether the better-fitting *no-rules* endpoint would react differently.

5.9.6 EXTERNAL VALIDITY IS LIMITED TO ONE TERMINAL AND TIME WINDOW

The temporal hold-out is one later period from the same terminal and source systems. The 99.944 % trie coverage shows repeated paths in this period. It does not show transfer to another process. The results do not cover another terminal, season, gate regime, or infrastructure setup. That would require another full discovery and validation run.

6

Conclusion

This thesis investigated the applicability, historical fidelity, and limitations of automatically discovered data-aware process simulation for container-terminal truck operations. The work was motivated by the increasing availability of operational event data in container terminals and by recent advances in simulation discovery, which make it possible to derive executable simulation models directly from recorded process behavior. While these approaches have been studied primarily in conventional business-process settings, their applicability to physical logistics systems with spatial constraints, congestion effects, and resource dependencies has not yet been researched in depth.

To investigate this problem, the thesis developed a simulation-ready event log from two months of continuous operational truck-handling data of the HHLA Container Terminal Burchardkai and applied the ProSiT simulation-discovery framework. The resulting models were evaluated on a temporally held-out test period resembling unseen future from the perspective of the simulation models development. The evaluation combined control-flow conformance, distributional quality measures, repeated Monte-Carlo simulation, and explicit structural checks. Different levels of contextual model expressiveness were compared, and the final data-aware model was subsequently used in two transparent what-if experiments concerning an RMG resource-pool intervention and increased truck demand.

The implementation and empirical evaluation answer the four research questions introduced

in Chapter 1.

RQ1 questions, to what extent a ProSiT simulation model discovered from historical CTB truck-operation data can reproduce the behaviour observed in an unseen temporal hold-out. The evaluation showed that the model reproduces some patterns of activities and process characteristics of the observed temporal behavior considerably better than others. The final baseline achieves an unweighted yard-service Wasserstein distance of 2.99 minutes and a real-frequency-weighted distance of 2.34 minutes, while the inter-arrival distribution differs from the held-out log by only 0.072 minutes. Despite these fairly accurate statistical results, the model systematically under-predicts complete truck turnaround time. Mean turnaround is 30.44 minutes in the simulation compared with 39.74 minutes in the held-out data, corresponding to an under-prediction of 9.31 minutes. [Add confidence Intervall Results and interpretation]

RQ2 investigated the suitability of automatically discovered control flow for the CTB truck process. An Inductive-Miner model was initially discovered in line with the ProSiT methodology, but its generalised structure introduced concurrency and ordering behaviour that conflicted with known physical constraints of a single truck visit. Consequently, the discovered model could not be used unchanged as the simulation backbone. The final implementation therefore replaced it with a deliberately restrictive prefix-trie Petri net derived from the observed training variants. This representation covered 99.944 % of held-out traces, but its high coverage should not be interpreted as superior process discovery because the model intentionally sacrifices generalisation. The main finding of RQ2 is therefore that transferring process-discovery-based simulation to the CTB setting required explicit domain constraints on the control-flow model; automatic process discovery alone did not produce a sufficiently suitable simulation structure.

RQ3 asked how increasing the contextual expressiveness of the simulation model affects held-out fidelity. The comparison between the distribution-only *no-rules* configuration and the full *rules+workload* configuration shows that additional data-aware modelling does not automatically improve simulation fidelity. The more expressive configuration produces a 56 % higher turnaround-distance measure and greater variation across simulation seeds, while its improvement in inter-arrival fidelity is operationally negligible. A subsequent per-activity analysis shows that the available contextual attributes explain little of the service-time variation of the dominant RMG activities. The results therefore indicate that increasing model expressiveness is only useful when the available contextual information captures variation that is relevant to the simulated outcome. In the CTB event log examined here, additional contextual conditioning

introduces modelling complexity without producing a corresponding improvement in overall held-out fidelity.

RQ4 asked how the discovered simulation model responds to transparent operational interventions and what these responses reveal about its suitability for what-if decision support. Both interventions can be implemented, inspected, and reproduced consistently. Removing T22 from the simulated RMG resource pool eliminates all assignments to that resource, while increasing the working-time arrival intensity by 20 % produces the intended change in the underlying arrival parameter. Nevertheless, neither intervention produces a statistically resolved change in mean truck turnaround time. More importantly, the scenario analysis exposes limitations of the underlying process abstraction. RMG resources represent statistical capacity assignments rather than physical container locations, and the model does not represent container relocation, equipment movement, waterside and landside crane allocation, or explicit spatial queue states. Consequently, the scenarios are valid experiments on the discovered simulation model, but their results cannot be interpreted as experimentally established counterfactual effects of the physical terminal.

Overall, the thesis shows that automatically discovered data-aware process simulation can be transferred to container-terminal truck operations and can produce a transparent, reproducible, and structurally consistent simulation model from operational event data. At the same time, the case study demonstrates that successful process discovery, historical fidelity, and scenario executability are different requirements. High control-flow conformance does not guarantee accurate temporal behaviour, additional contextual features do not necessarily improve simulation fidelity, and an executable what-if intervention does not establish that the corresponding physical effect is represented correctly.

The contribution of this thesis is therefore not a demonstration that the discovered model is already sufficient for unrestricted operational decision support. Rather, it provides empirical evidence about where a recent white-box simulation-discovery approach transfers successfully to a complex logistics environment and where the abstraction reaches its limits. For the CTB truck process, the results indicate that further progress towards trustworthy operational decision support requires richer representations of spatial process state, physical resource constraints, equipment movement, and queueing mechanisms in addition to the event-log-based behavioural information captured by the current simulation model.

6.1 FUTURE WORK

[TBD] *Researching alternative simulation approaches to the classical manual simulation of container terminal logistics is an interesting and in my opinion fruitful field. Due to the 24/7 operations in logistics, the resources for development and research in advanced technologies is very limited and I see great potential in digital start ups or tech companies to get in a foot on that. For port applications, alternative eventlog structure with different aggregation logic or Event specific attributes kept and a vessel and rail arrival logic implemented.*

The framework developed and applied in this Thesis can be extended in the following areas:

- Alternative Simulation Parametrisation: Tbd
- Alternative Terminal Setting: An alternative simulation scenario to be analysed, like the Quaysides workload effects.
- Different Simulation Tool Due to the physical component, the application of a business process tool has its limits in logistics application.
- Alternative Event log Structure The Attribute selection and aggregation affect the results heavily. An alternative engineering or retainig attributes on the event level could also affect simulation outcomes, which could be investigated in future simulation scenarios.

References

- [1] W. Y. Yun and Y. S. Choi, "A simulation model for container-terminal operation analysis using an object-oriented approach," *International Journal of Production Economics*, vol. 59, no. 1, pp. 221–230, 1999. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925527398002138>
- [2] D. Steenken, S. Voss, and R. Stahlbock, "Container terminal operation and operations research - a classification and literature review," *OR Spectrum*, vol. 26, pp. 3–49, 01 2004.
- [3] B. Weerasinghe, N. Perera, and X. Bai, "Optimizing container terminal operations: a systematic review of operations research applications," *Maritime Economics and Logistics*, vol. 26, pp. 1–35, 02 2023.
- [4] D. Bett, I. Ali, M. Gheith, and A. Eltawil, "Simulation-based optimization of truck appointment systems in container terminals: A dual transactions approach with improved congestion factor representation," *Logistics*, vol. 8, p. 80, 08 2024.
- [5] A. Rozinat, R. Mans, M. Song, and W. Aalst, "Discovering simulation models," *Information Systems*, vol. 34, pp. 305–327, 05 2009.
- [6] V. Leno, M. Dumas, F. Maggi, M. La Rosa, and A. Polyvyanyy, "Automated discovery of declarative process models with correlated data conditions," *Information Systems*, vol. 89, p. 101482, 12 2019.
- [7] F. Mannhardt, S. J. J. Leemans, C. T. Schwanen, and M. de Leoni, "Modelling data-aware stochastic processes - discovery and conformance checking," in *Application and Theory of Petri Nets and Concurrency: 44th International Conference, PETRI NETS 2023, Lisbon, Portugal, June 25–30, 2023, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 77–98. [Online]. Available: https://doi.org/10.1007/978-3-031-33620-1_5
- [8] M. {de Leoni}, F. Vinci, S. Leemans, and F. Mannhardt, "Investigating the influence of data-aware process states on activity probabilities in simulation models: Does accuracy

- improve?” in *Business Process Management*, ser. Lecture Notes in Computer Science (LNCS), C. {Di Francescomarino}, A. Burattin, C. Janiesch, and S. Sadiq, Eds. Germany: Springer, Sep. 2023, pp. 129–145, 21st International Conference on Business Process Management, BPM 2023, BPM 2023 ; Conference date: 11-09-2023 Through 15-09-2023.
- [9] O. López-Pintado, S. Murashko, and M. Dumas, “Discovery and simulation of data-aware business processes,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.13666>
 - [10] F. Meneghello, C. Di Francescomarino, C. Ghidini, and M. Ronzani, “Runtime integration of machine learning and simulation for business processes: Time and decision mining predictions,” *Information Systems*, vol. 128, p. 102472, 10 2024.
 - [11] F. Vinci, G. Park, W. Aalst, and M. de Leoni, *Reliable and Configurable Process Simulations via Probabilistic White-Box Models*, 01 2026, pp. 337–352.
 - [12] F. Vinci, D. Aloini, E. Benevento, A. Stefanini, F. Zen, and M. de Leoni, “Improving organizational processes in healthcare through simulation-driven resource allocation: Methodology and real-world case study,” *Artificial Intelligence in Medicine*, vol. 176, p. 103391, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0933365726000436>
 - [13] M. Camargo, M. Dumas, and O. González-Rojas, “Automated discovery of business process simulation models from event logs,” *Decision Support Systems*, vol. 134, p. 113284, 03 2020.
 - [14] S. Hartmann, *Generating scenarios for simulation and optimization of container terminal logistics*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 101–122. [Online]. Available: https://doi.org/10.1007/3-540-26686-0_4
 - [15] Hamburger Hafen und Logistik AG, “Geschäftsbericht 2025,” Hamburger Hafen und Logistik AG, Hamburg, Tech. Rep., 2026. [Online]. Available: https://bericht.hhla.de/geschaeftsbericht-2025/_assets/downloads/HHLA_2025_Geschaeftsbericht.pdf
 - [16] M. Gracia, J. Mar-Ortiz, C. Caballini, and S. Sacone, “A combined data mining – optimization approach to manage trucks operations in container terminals with the use of a tas: Application to an italian and a mexican port,” *Transportation Research Part E Logistics and Transportation Review*, vol. 142, 08 2020.

- [17] M. Kastner, N. Nellen, A. Schwientek, and C. Jahn, "Integrated simulation-based optimization of operational decisions at container terminals," *Algorithms*, vol. 14, 01 2021.
- [18] A. Carboni, F. Defflorio, C. Caballini, and S. Cangelosi, "Advances in terminal management: Simulation of vehicle traffic in container terminals," *Maritime Economics & Logistics*, vol. 27, no. 3, pp. 500–524, 2025. [Online]. Available: <https://doi.org/10.1057/s41278-024-00300-5>
- [19] D. Muravev, A. Rakhmangulov, H. Hu, and H. Zhou, "The introduction to system dynamics approach to operational efficiency and sustainability of dry port's main parameters," *Sustainability*, vol. 11, p. 2413, 04 2019.
- [20] D. Muravev, H. Hu, A. Rakhmangulov, and P. Mishkurov, "Multi-agent optimization of the intermodal terminal main parameters by using anylogic simulation platform: Case study on the ningbo-zhoushan port," *International Journal of Information Management*, vol. 57, p. 102133, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S026840121931789X>
- [21] W. Van der Aalst, *Process Mining: Data Science in Action*, 01 2016.
- [22] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs containing infrequent behaviour," in *Business Process Management Workshops*, ser. Lecture Notes in Business Information Processing, vol. 171. Springer, 2014, pp. 66–78.
- [23] R. de la Briandais, "File searching using variable length keys," in *Proceedings of the Western Joint Computer Conference*. ACM, 1959, pp. 295–298.
- [24] S. A. Fahrenkrog-Petersen, H. van der Aa, and M. Weidlich, "Pretsa: Event log sanitization for privacy-aware process discovery," in *2019 International Conference on Process Mining (ICPM)*. IEEE, 2019, pp. 1–8.
- [25] W. Aalst, A. Adriansyah, and B. Dongen, "Replaying history on process models for conformance checking and performance analysis," *WIREs Data Mining and Knowledge Discovery*, vol. 2, pp. 182–192, 03 2012.
- [26] A. Adriansyah, B. F. van Dongen, and W. M. P. van der Aalst, "Conformance checking using cost-based fitness analysis," in *2011 IEEE 15th International Enterprise Distributed Object Computing Conference*. IEEE, 2011, pp. 55–64.

- [27] W. M. P. van der Aalst, “Business process simulation survival guide,” in *Handbook on Business Process Management 1: Introduction, Methods, and Information Systems*, 2nd ed., J. vom Brocke and M. Rosemann, Eds. Berlin, Heidelberg: Springer, 2015, pp. 337–370.
- [28] A. Bruzzone, F. Longo, L. Nicoletti, E. Bottani, and R. Montanari, “Simulation, analysis and optimization of container terminals processes,” *International Journal of Modeling, Simulation, and Scientific Computing*, vol. 03, p. 1240006, 11 2012.
- [29] D. Chapela-Campa, M. Dumas, O. Pintado, and I. Suvorau, “Simod: Automated discovery of business process simulation models,” 01 2024.
- [30] A. Solti, W. Aalst, and M. Weske, “Discovering stochastic petri nets with arbitrary delay distributions from event logs,” vol. 171, 08 2013.
- [31] M. Camargo, M. Dumas, and O. González-Rojas, “Discovering generative models from event logs: data-driven simulation vs deep learning,” *PeerJ Computer Science*, vol. 7, p. e577, 07 2021.
- [32] A. Burke, S. Leemans, and M. Wynn, “Stochastic process discovery by weight estimation,” 10 2020.
- [33] F. Vinci and M. de Leoni, “Balancing fitness and precision in process model repair: Framework formalization, assessment, and benefits for simulation,” *Process Science*, vol. 2, no. 1, p. 3, 2025. [Online]. Available: <https://doi.org/10.1007/s44311-025-00007-7>
- [34] O. Pintado and M. Dumas, *Business Process Simulation with Differentiated Resources: Does it Make a Difference?*, 09 2022, pp. 361–378.
- [35] R. G. Sargent, “Verification and validation of simulation models,” *Journal of Simulation*, vol. 7, no. 1, pp. 12–24, 2013.
- [36] D. Chapela-Campa, I. Benchekroun, O. Baron, M. Dumas, D. Krass, and A. Senderovich, “Can i trust my simulation model? measuring the quality of business process simulation models,” in *Business Process Management*, ser. Lecture Notes in Computer Science, vol. 14159. Cham: Springer, 2023, pp. 20–37.
- [37] —, “A framework for measuring the quality of business process simulation models,” *Information Systems*, vol. 127, p. 102447, 01 2025.

- [38] S. J. J. Leemans, A. F. Syring, and W. M. P. van der Aalst, “Earth movers’ stochastic conformance checking,” in *Business Process Management Workshops*, ser. Lecture Notes in Business Information Processing, vol. 360. Cham: Springer, 2019, pp. 127–143.
- [39] J. Massey, Frank J., “The kolmogorov–smirnov test for goodness of fit,” *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.
- [40] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York: McGraw-Hill Education, 2015.

Acknowledgments

TBD